Memory and Data Locality

Objectives

- Learn how to effectively use the CUDA memory types
  - Performance impact by memory access efficiency
  - Scope and lifetime of different memories
- Understand the motivation and ideas for tiled parallel algorithms
- Understand the roofline performance model
- Case study: matrix multiplication
  - Learn to write a tiled matrix-multiplication kernel
  - Handle arbitrary matrix sizes in tiled matrix multiplication

Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {
  // Calculate the row index of the P element and M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

  // Calculate the column index of P and N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < Width; ++k) {
      Pvalue += M[Row*Width+k]*N[k*Width+Col];
    }
    P[Row*Width+Col] = Pvalue;
  }
}
```

Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {

  // Calculate the row index of the P element and M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

  // Calculate the column index of P and N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < Width; ++k) {
      Pvalue += M[Row*Width+k]*N[k*Width+Col];
    }
    P[Row*Width+Col] = Pvalue;
  }
}
```

Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {

  // Calculate the row index of the P element and M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

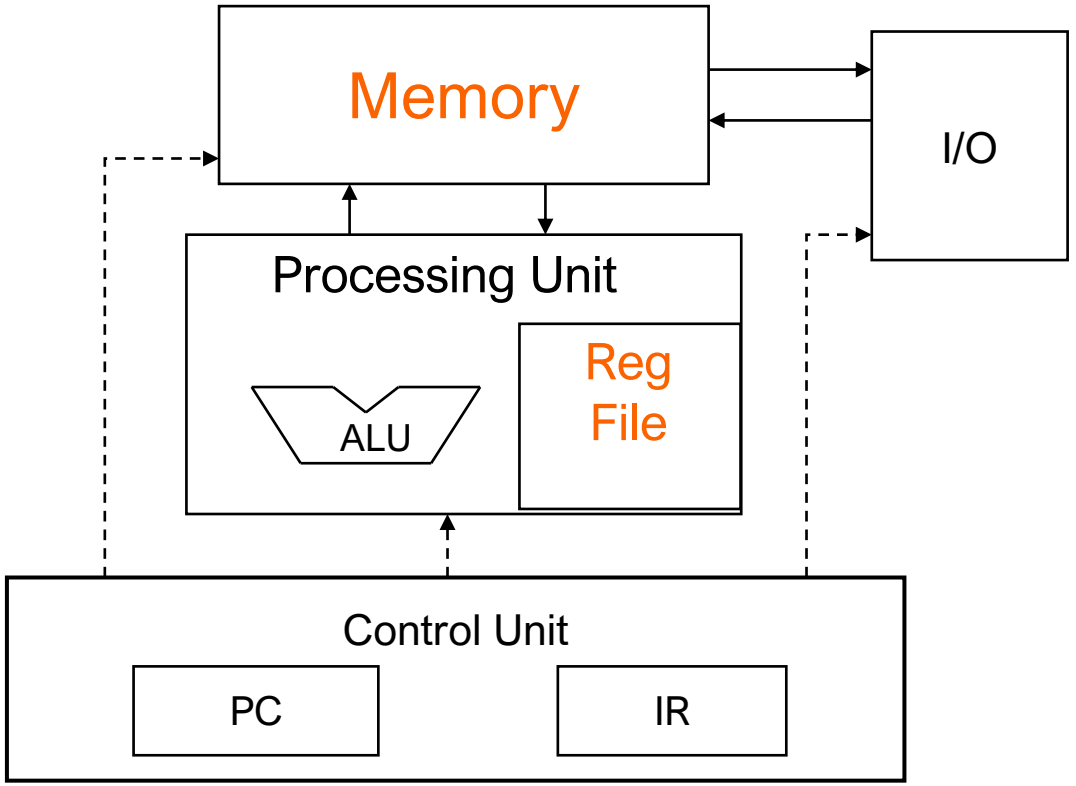
  // Calculate the column index of P and N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < Width; ++k) {
      Pvalue += M[Row*Width+k]*N[k*Width+Col];
    }
    P[Row*Width+Col] = Pvalue;
  }
}
```

Performance on a GPU

- All threads access global memory for their input matrix elements
  - Two memory accesses (8 bytes) for floating-point multiplication and addition
  - 4B/s of memory bandwidths/Flops
- Assume a GPU with
  - Peak floating-point rate 1.6 TFlops with 600GB/s DRAM bandwidth
  - 4*1.6 = 6.4 TB/s required to achieve peak FLOPS rating
  - The 600GB/s memory bandwidth limits the execution at 150 Gflops
- This limits the execution rate to 9.3% (0.15/1.6) of the peak floating-point execution rate of the device
  - C.f., 1.7% for H100 (67 TFlops, 4.8TB/s)
- **Need to drastically cut down memory accesses to get close to the 1.6 TFlops**

Memory and Registers in the Von-Neumann Model

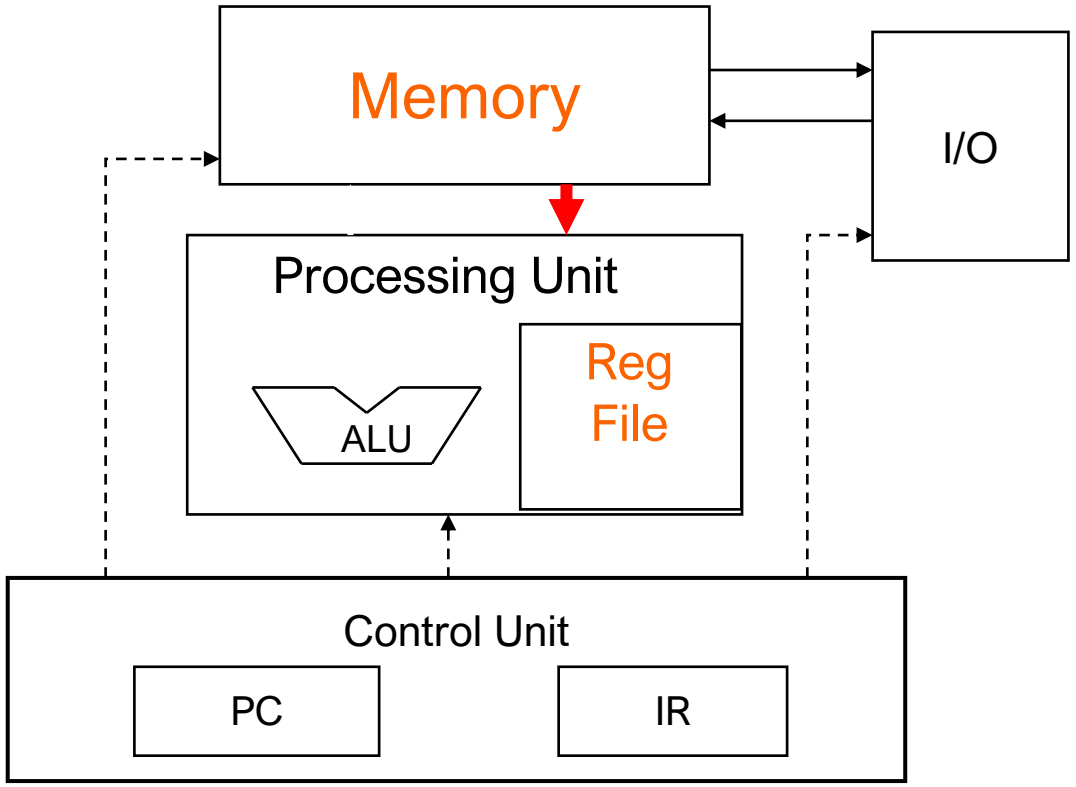


Memory and Registers in the Von-Neumann Model

Memory instructions (e.g., LD r4, r6)
- Load the contents of memory at the address @ r6
- Write the contents to R4

Compute operations (e.g., add r1, r2, r3)
- Read r2 and r3
- Add them
- Write the result to r1

To perform any compute operation, data should be loaded first from memory to registers



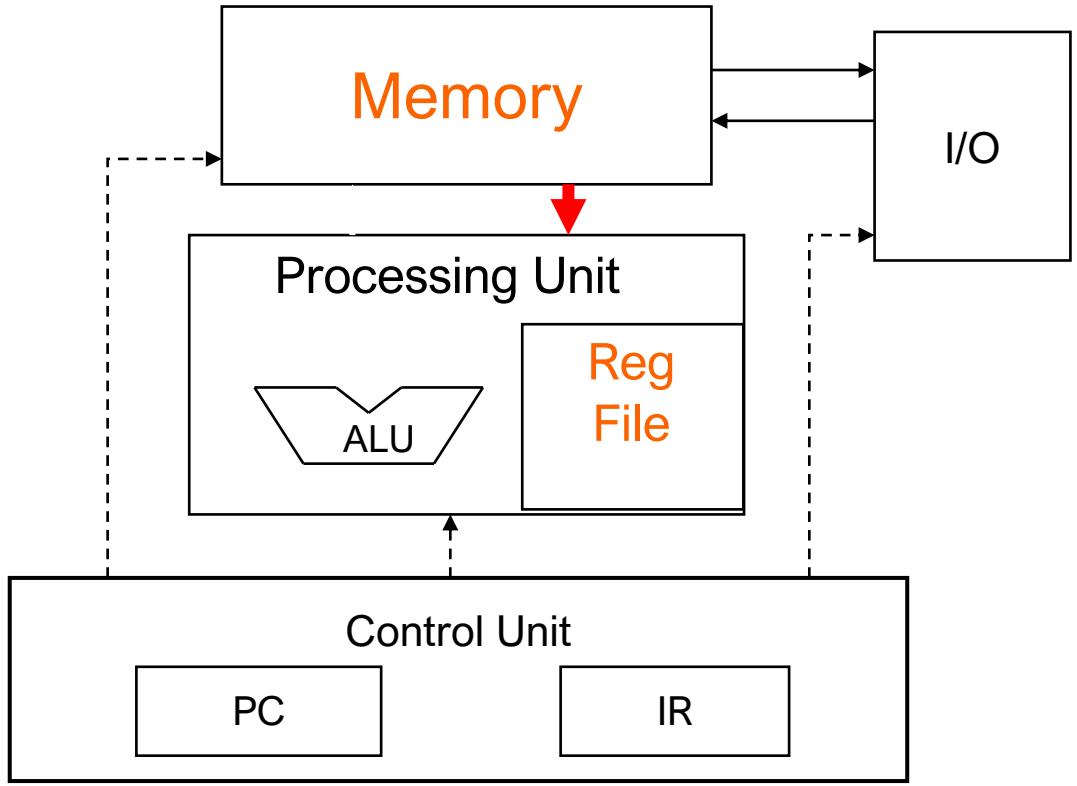
Registers vs. Memory

Registers

- Fast: 1 cycle; no memory access required
- Scarce: hundreds for CPU, O(10k) for GPU SM

Memory

- Slow: hundreds of cycles
- Huge: GBs or more



Registers vs. Memory

Registers

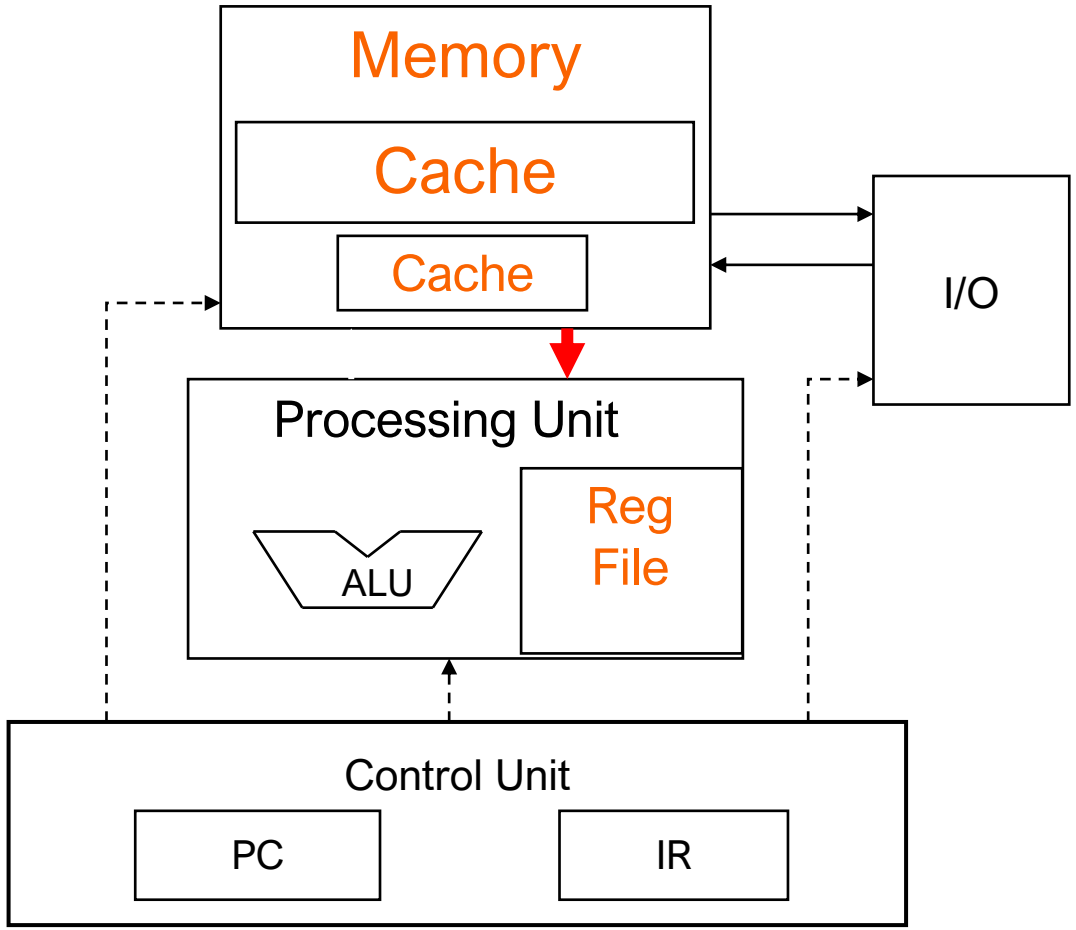
- Fast: 1 cycle; no memory access required
- Scarce: hundreds for CPU, O(10k) for GPU SM

Caches

- Faster: 10-100 cycles; hierarchical
- Smaller: KB to MB

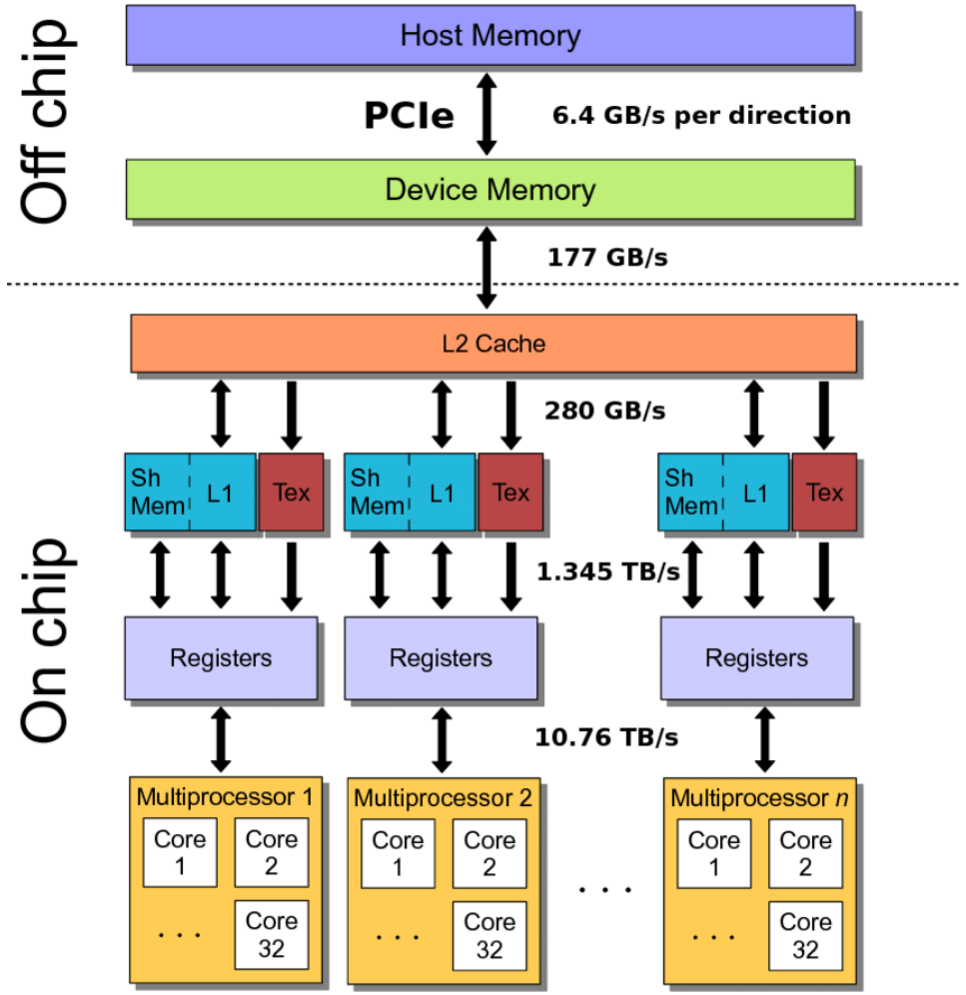
Memory

- Slow: hundreds of cycles
- Huge: GBs or more



CUDA Memory Hierarchy

- Each thread can read/write
  - Per thread **register** s(1-2 cycles)
  - Per thread **local memory** (~500 cycles)
  - Per-thread block **shared memory** (10s of cycles)
  - Per-grid **global memory** (~500 cycles)
- Each thread can read only
  - Per-grid **constant and texture memory**
    (100s cycles for cache hits)
- For global memory, cache hits take
  - 10s cycles for L1 (texture and constant caches)
  - 100s cycles for L2



Shared Memory in GPUs

◎ A special type of memory whose contents are explicitly defined and used in the kernel source code

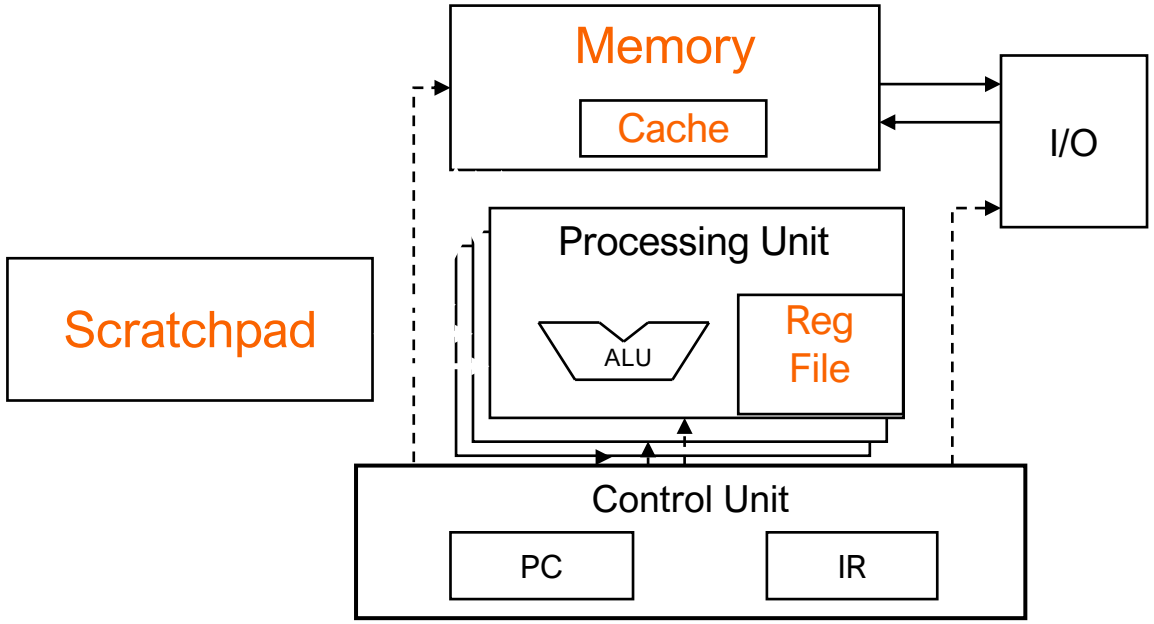
- One in each SM/CU in both NVIDIA and AMD GPUs
- Scope of access and sharing: thread blocks
- Lifetime: thread block
  - Contents will disappear after the corresponding thread block finishes execution
- Explicitly accessed by memory load/store instructions
- A.k.a. “scratchpad memory”

Cache vs. Shared Memory (Scratchpad)

◎ **Cache**

- HW-managed (Transparent to the SW)
- Power overhead from tag matching
- Implicit data movement
- May result in waste of memory

- Scratchpad
  - SW-managed (separate address space)
  - Directly addressed; Lower power consumption
  - Explicit data movement
  - Compact and efficient storage



CUDA Variable Type Qualifiers

- __device__ is optional when used with __shared__, or __constant__

- "automatic" scalar variables without qualifier reside in a register
  - compiler will spill to thread local memory

- "automatic" array variables without qualifier reside in thread-local memory

Variable declaration		Memory	Scope	Lifetime
	`int LocalVar;`	register	thread	thread
`__device__ __shared__`	`int SharedVar;`	shared	block	block
`__device__`	`int GlobalVar;`	global	grid	application
`__device__ __constant__`	`int ConstantVar;`	constant	grid	application

CUDA Variable Type Latency

- Scalar variables reside in fast, on-chip registers
- Shared variables reside in fast, on-chip memories
- Thread-local arrays, global variables, and constant variables reside in off-chip DRAM (can be cached)

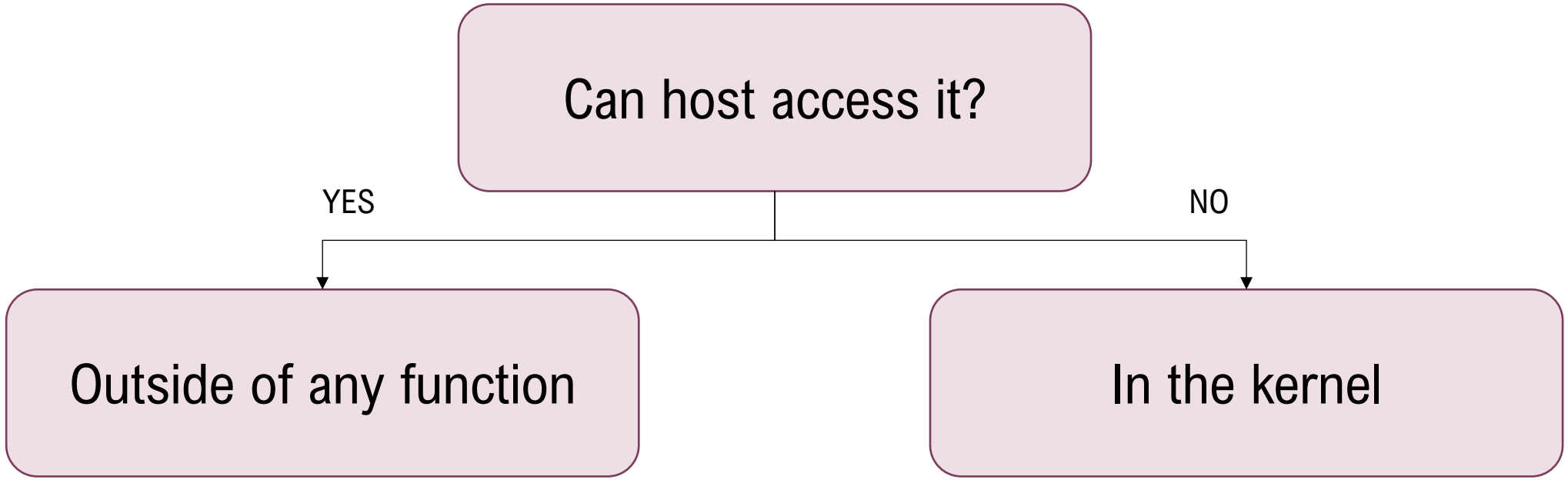
Variable declaration	Memory	Penalty
`int LocalVar;`	register	1x
`int LocalVar[10];`	local	10-100x
`__shared__ int SharedVar;`	shared	10x
`__device__ int GlobalVar;`	global	10-100x
`__constant__ int ConstantVar;`	constant	10-100x

CUDA Variable Type Scale

- 100Ks per-thread variables, R/W by 1 thread
- 100s shared variables, each R/W by 100s of threads
- 1 global variable is R/W by 100Ks threads
- 1 constant variable is readable by 100Ks threads

Variable declaration		Instances	Visibility
	`int LocalVar;`	100,000s	1
	`int LocalVar[10];`	100,100s	1
`__shared__`	`int SharedVar;`	100s	100s
`__device__`	`int GlobalVar;`	1	100,000s
`__constant__`	`int ConstantVar;`	1	100,000s

Where to Declare Variables?




```
__constant__ int ConstantVar;
__device__ int GlobalVar;
```

```
int LocalVar;
int LocalVar[10];
__shared__ int SharedVar;
```

Programming Strategy with Memory Types

- Strategically place data according to access patterns

- Read-only → constant memory (fast)
  - Good for small (less than 64KB) read-only data shared by many threads
- R/W and shared within a block → local (shared) memory (fast)
- R/W within each thread → registers (fast)

- Indexed R/W within each thread → thread-local memory (slow)
- R/W inputs/results → global memory (slow)

Example: Thread-local Variables

```
// motivate per-thread variables with Ten Nearest Neighbors application
__global__ void ten_nn(float2 *result,
                       float2 *ps,
                       float2 *qs,
                       size_t num_qs) {
    // p goes in a register
    float2 p = ps[threadIdx.x];
    // per-thread heap goes into off-chip memory
    float2 heap[10];
     // read through num_qs points, maintaining
     // the nearest 10 qs to p in the heap
    ...
    // write out the contents of heap to result
    ...
}
```

Examples: Shared Memory

```
// motivate shared variables with Adjacent Difference application
// compute result[i] = input[i] - input[i-1]
  __global__ void adj_diff_naive(int *result, int *input) {
      // compute this thread's global index
      unsigned int i = blockDim.x * blockIdx.x + threadIdx.x;

      if (i > 0) {
        // each thread loads two elements from global memory
        int x_i = input[i];
        int x_i_minus_one = input[i-1];

        result[i] = x_i - x_i_minus_one;
      }
}
```

Examples: Shared Memory

```
// motivate shared variables with Adjacent Difference application
// compute result[i] = input[i] - input[i-1]
__global__ void adj_diff_naive(int *result, int *input) {
        // compute this thread's global index
        unsigned int i = blockDim.x * blockIdx.x + threadIdx.x;

        if (i > 0) {
            // how many times does this kernel load input[i]?
            int x_i = input[i];
            int x_i_minus_one = input[i-1];

            result[i] = x_i - x_i_minus_one;
        }
}
```

Examples: Shared Memory

```
// motivate shared variables with Adjacent Difference application
// compute result[i] = input[i] - input[i-1]
__global__ void adj_diff_naive(int *result, int *input) {
        // compute this thread's global index
        unsigned int i = blockDim.x * blockIdx.x + threadIdx.x;

        if (i > 0) {
           // how many times does this kernel load input[i]?
           int x_i = input[i]; // once by thread i
           int x_i_minus_one = input[i-1]; // once by thread i+1

           result[i] = x_i - x_i_minus_one;
        }
}
```

Examples: Shared Memory

```
// motivate shared variables with Adjacent Difference application
// compute result[i] = input[i] - input[i-1]
__global__ void adj_diff_naive(int *result, int *input) {
        // compute this thread's global index
        unsigned int i = blockDim.x * blockIdx.x + threadIdx.x;

        if (i > 0) {
           // how many times does this kernel load input[i]?
           int x_i = input[i]; // once by thread i
           int x_i_minus_one = input[i-1]; // once by thread i+1

           result[i] = x_i - x_i_minus_one;
        }
}
```

Eliminate redundant global accesses by sharing data

Barrier Synchronization

- A built-in function in CUDA
  - `__syncthreads()`

- All threads in the same thread block must reach the barrier before any thread can move on

- Used to ensure data is ready for communication
  - Demarcates “phase boundaries”

Example: Shared Memory

```
// optimized version of adjacent difference
__global__ void adj_diff(int *result, int *input)
{
  int tx = threadIdx.x; // shorthand for threadIdx.x

  // allocate a __shared__ array, one element per thread
  __shared__ int s_data[BLOCK_SIZE];
  // each thread reads one element to s_data
  unsigned int i = blockDim.x * blockIdx.x + tx;
  s_data[tx] = input[i];

  // avoid race condition: ensure all loads
  // complete before continuing
  __syncthreads();
  ...
```

Example: Shared Memory

```
// optimized version of adjacent difference
__global__ void adj_diff(int *result, int *input)
{
  int tx = threadIdx.x; // shorthand for threadIdx.x

  // allocate a __shared__ array, one element per thread
  __shared__ int s_data[BLOCK_SIZE];
  // each thread reads one element to s_data
  unsigned int i = blockDim.x * blockIdx.x + tx;
  s_data[tx] = input[i];

  // avoid race condition: ensure all loads
  // complete before continuing
  __syncthreads();
  ...
```

Example: Shared Memory

```
...
  if (tx > 0)
    result[i] = s_data[tx] - s_data[tx-1];
  else if (i > 0)
  {
    // handle thread block boundary
    result[i] = s_data[tx] - input[i-1];
  }
}
```

Example: Dynamic Allocation of Shared Memory

```
// when the size of the array isn't known at compile time...
__global__ void adj_diff(int *result, int *input)
{
  // use extern to indicate a __shared__ array will be
  // allocated dynamically at kernel launch time
  extern __shared__ int s_data[];
  ...
}

// pass the size of the per-block array, in bytes, as the third
// argument to the triple chevrons
adj_diff<<<num_blocks, block_size, block_size * sizeof(int)>>>(r,i);
```

Example: Dynamic Allocation of Shared Memory

- If you need to dynamically allocate multiple arrays in shared memory
  - Use extern __shared__ and empty bracket works only for a single array
  - Dynamically allocate a large array and use pointers into it to divide it into multiple arrays

```
extern __shared__ int s[];
int *integerData = s;                          // nI ints
float *floatData = (float*)&integerData[nI]; // nF floats
char *charData = (char*)&floatData[nF];       // nC chars
...

//kernel launch from host
myKernel<<<gridSize, blockSize,
nI*sizeof(int)+nF*sizeof(float)+nC*sizeof(char)>>>(...);
```

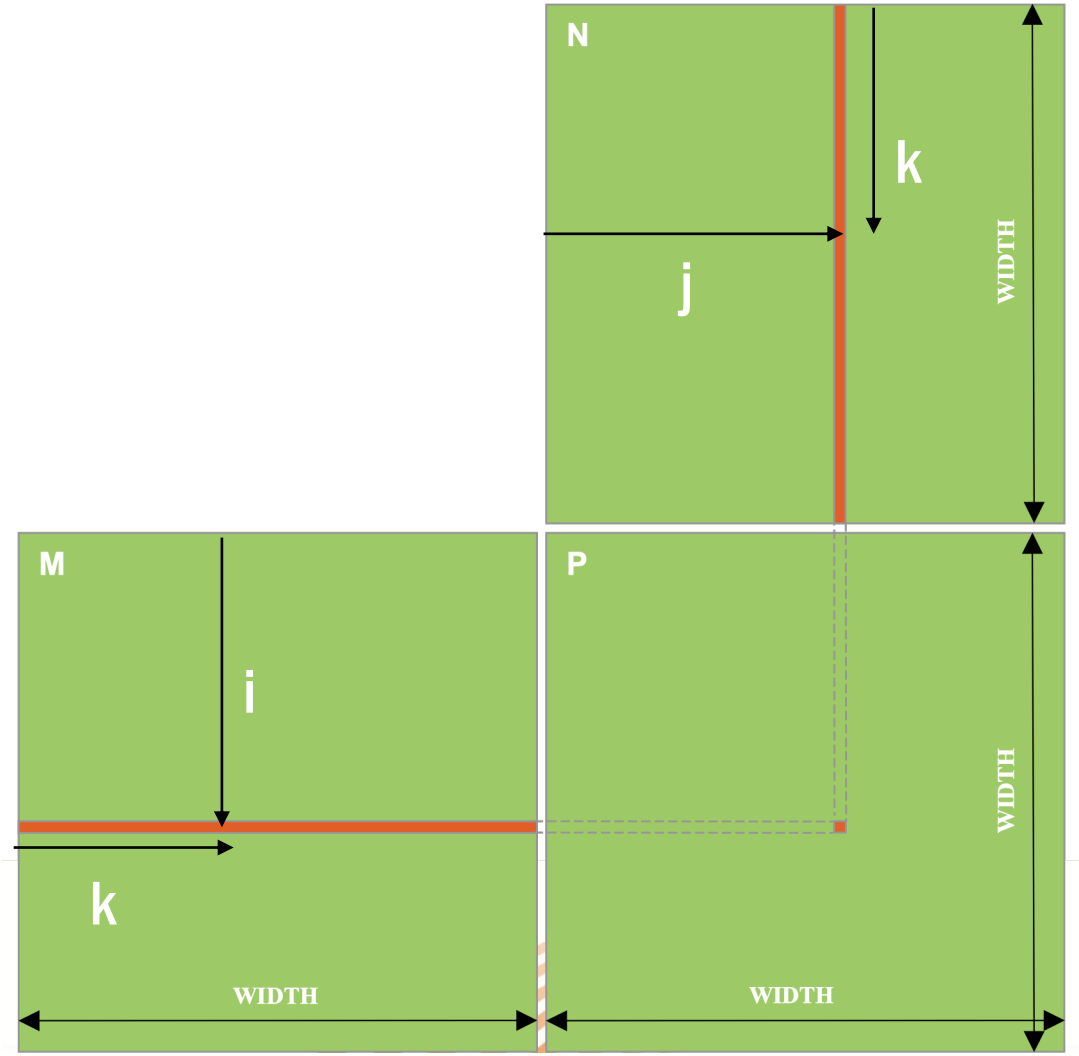

https://devblogs.nvidia.com/using-shared-memory-cuda-cc/

Tiling

Running Example

- Square matrix-matrix multiplication
  - Given two $WIDTH$ x $WIDTH$ matrices, M and N
  - $P = M \times N$
- In terms of the elements of P,

  compute $P_{ij} = \sum_{k=1}^{WIDTH} M_{ik} N_{kj}$



Running Example

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {

  // Calculate the row index of the P element and M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

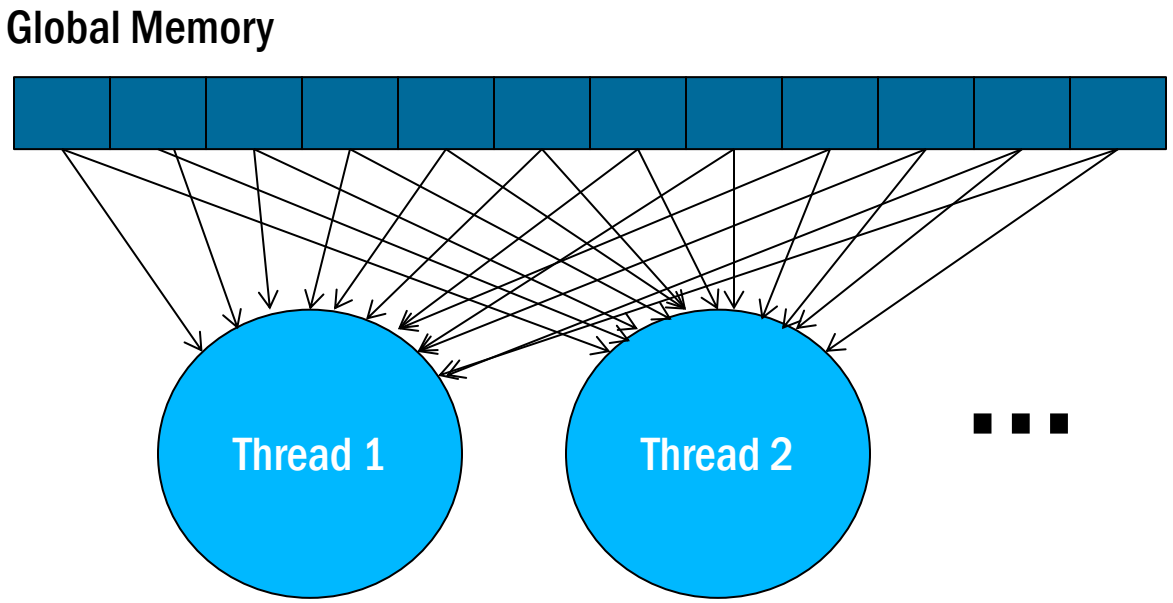
  // Calculate the column index of P and N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < Width; ++k) {
      Pvalue += M[Row*Width+k]*N[k*Width+Col];
    }
    P[Row*Width+Col] = Pvalue;
  }
}
```

Memory Access Pattern

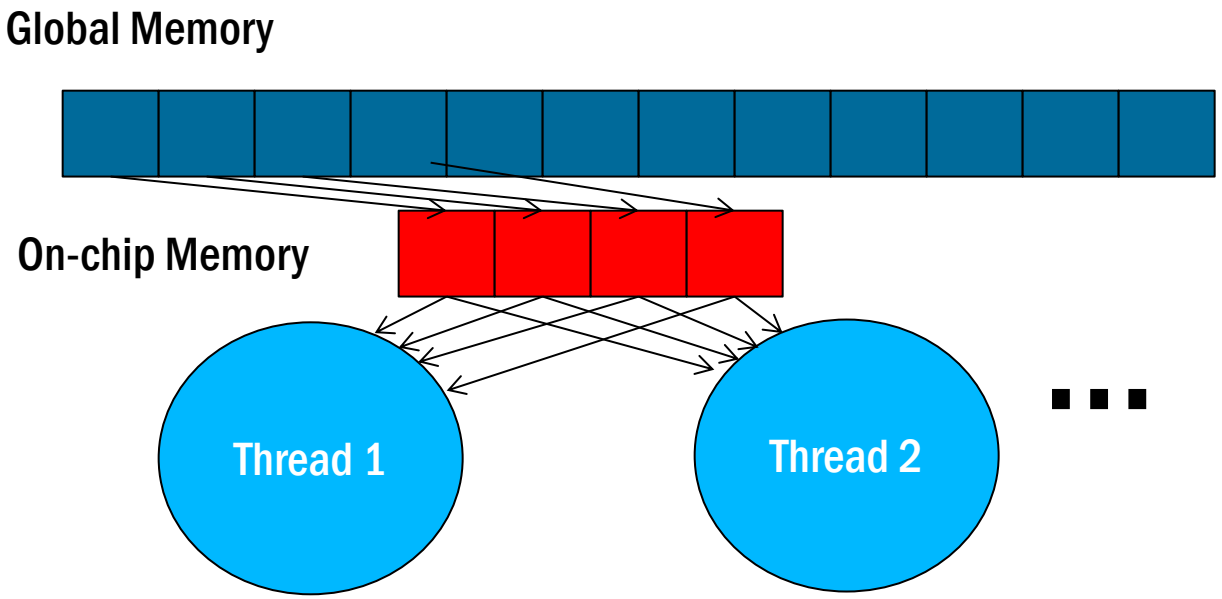
◎ Without tiling:

- One thread calculates one element of P
- M and N are loaded Width times from global memory by each thread



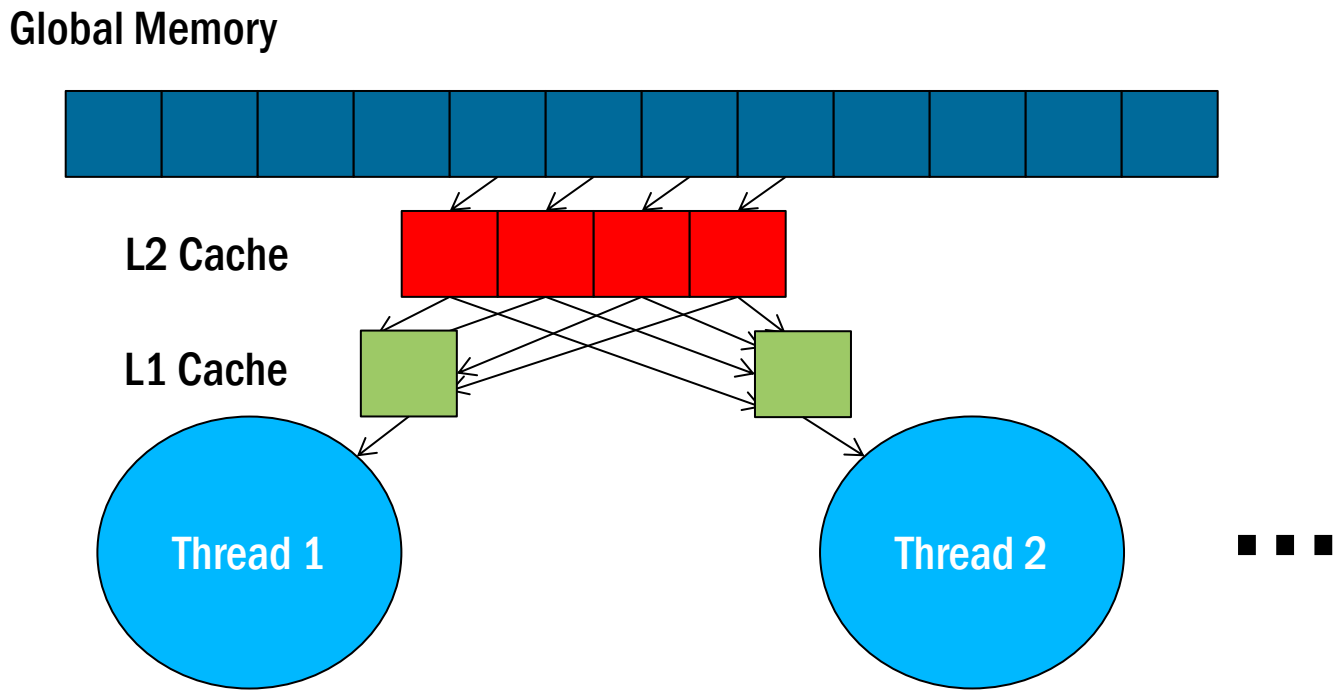
Tiling/Blocking

- Divide the global memory contents into “tiles”
- Focus the computation of threads on one or a smaller number of tiles at each point in time
- Keep the tile in faster on-chip memory



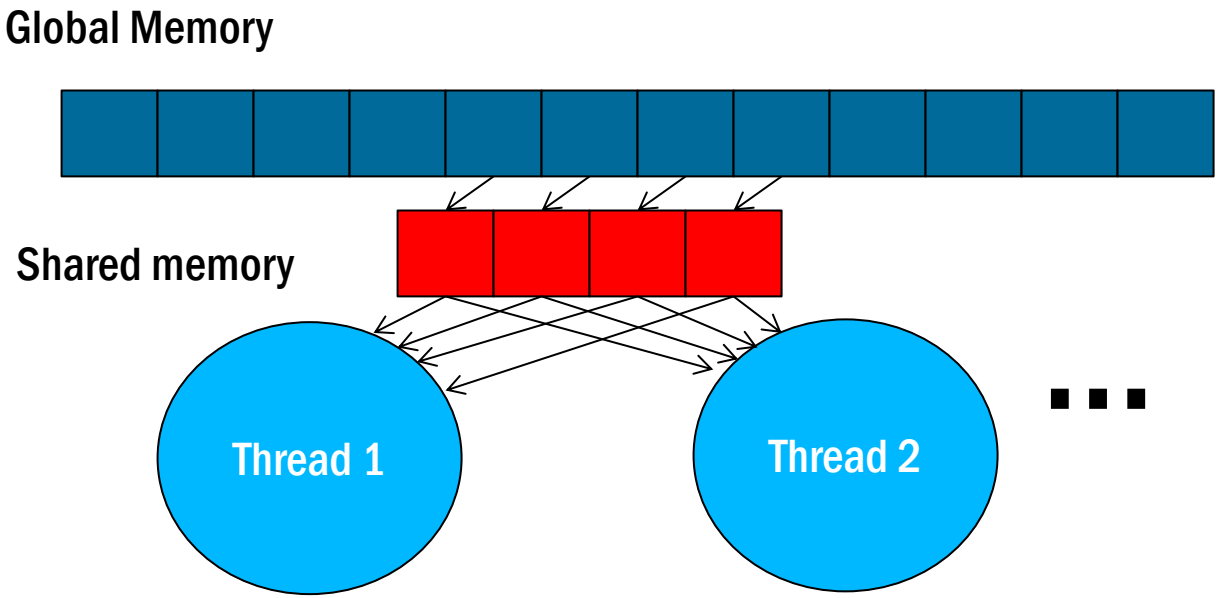
Tiling/Blocking

- Significantly reduce redundant off-chip memory access requests
- Can implicitly exploit caching effects from the memory hierarchy (L1 and L2)



Tiling/Blocking

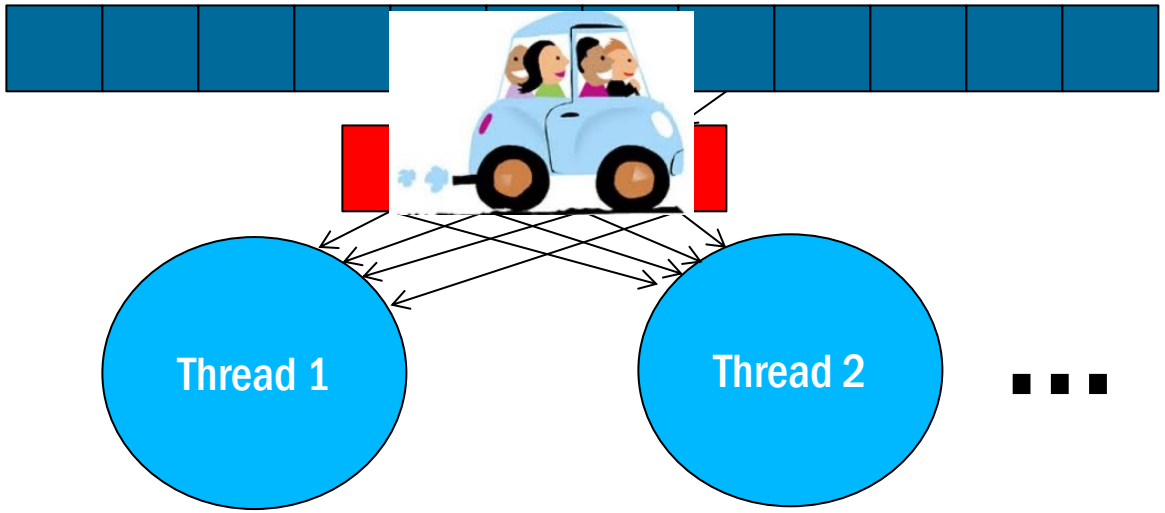
- Significantly reduce redundant off-chip memory access requests
- Can implicitly exploit caching effects from the memory hierarchy (L1 and L2)
- Or explicitly load tiles onto addressable memory (scratchpad)



Analogy

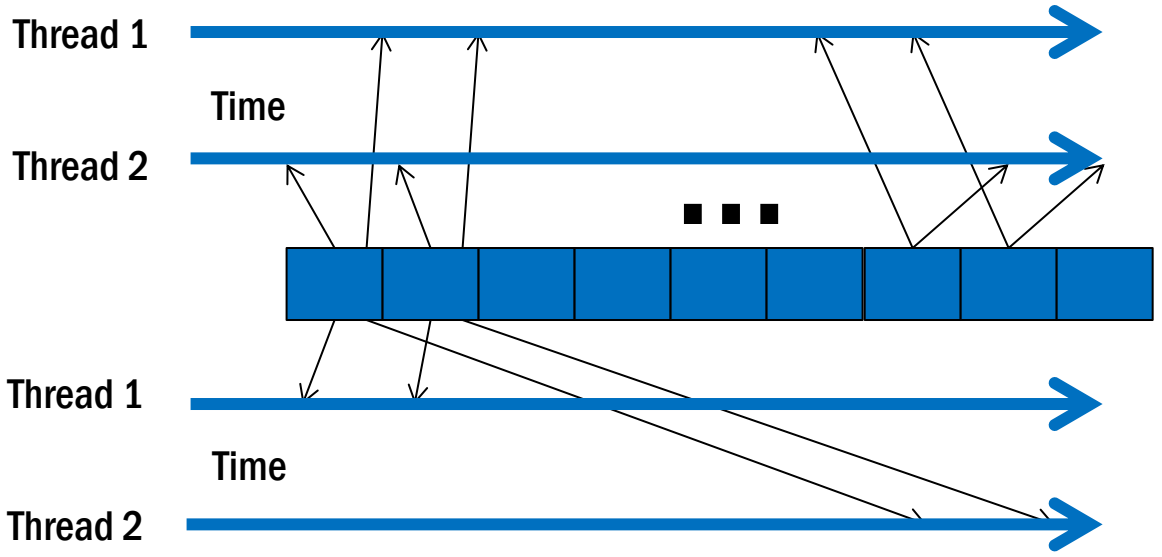
◎ In a congested traffic system, a reduced # of vehicles can greatly improve the delays seen by all vehicles

- Carpooling for commuters
- Tiling for global memory accesses
  - **Drivers** = threads accessing their data in memory
  - **Cars** = memory access requests



Analogy

- Carpools need synchronization
  - Works well and minimizes waiting time when people have similar schedule
- The same with tiling
  - Good when threads have similar access timing



Programming Strategy: Tiling

◎ Global memory resides in device memory (DRAM) which is much slower than on-chip shared memory

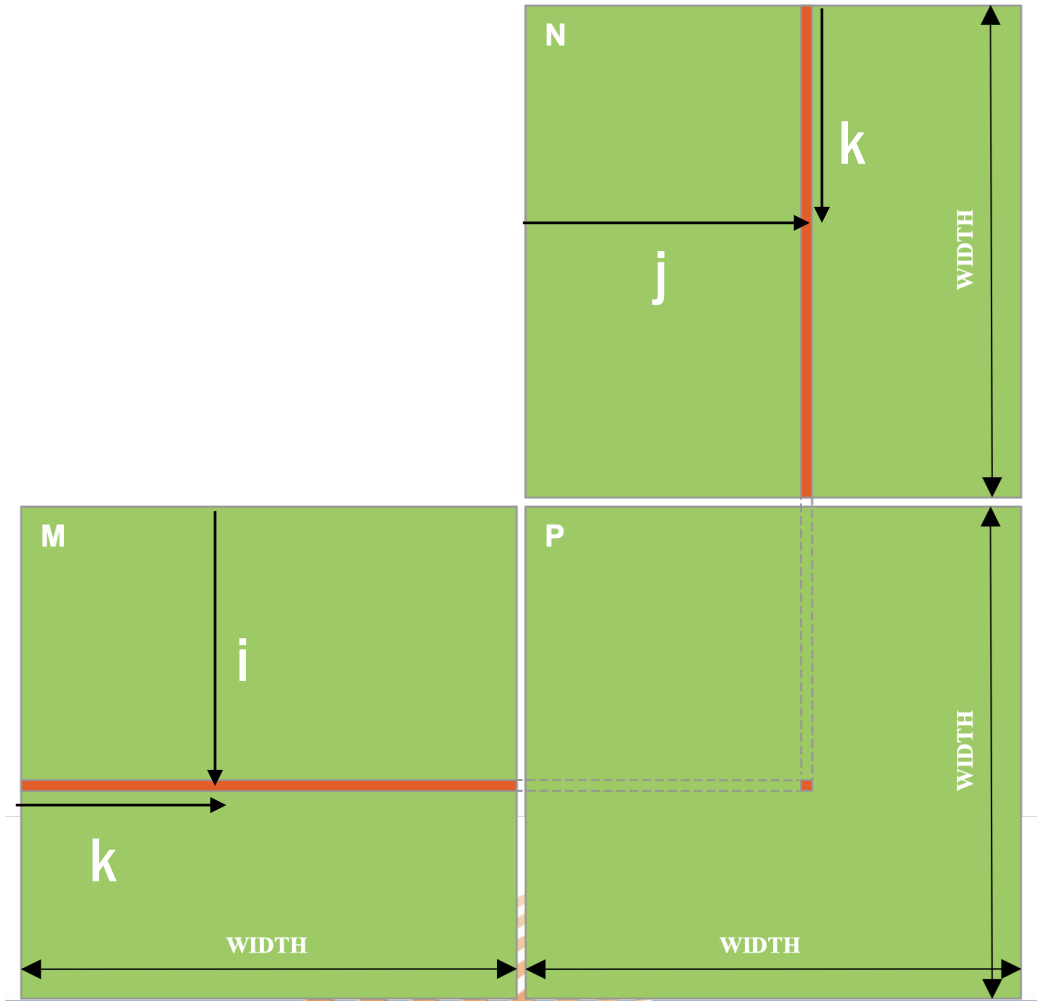
◎ **Tile data to take advantage of fast shared memory!**

1. Identify a ”tile” of global memory contents that are accessed by multiple threads and fit into shared memory
2. Load the tile from global memory into shared memory (using multiple threads)
3. Have the multiple threads to access their data from the on-chip memory and perform computation on them
4. Use barrier synchronizations to make sure data is ready
5. Copy results from shared memory to global memory
6. Move on to the next tile

Case Study: Matrix Multiplication

Matrix Multiplication: A Simple Host Version in C

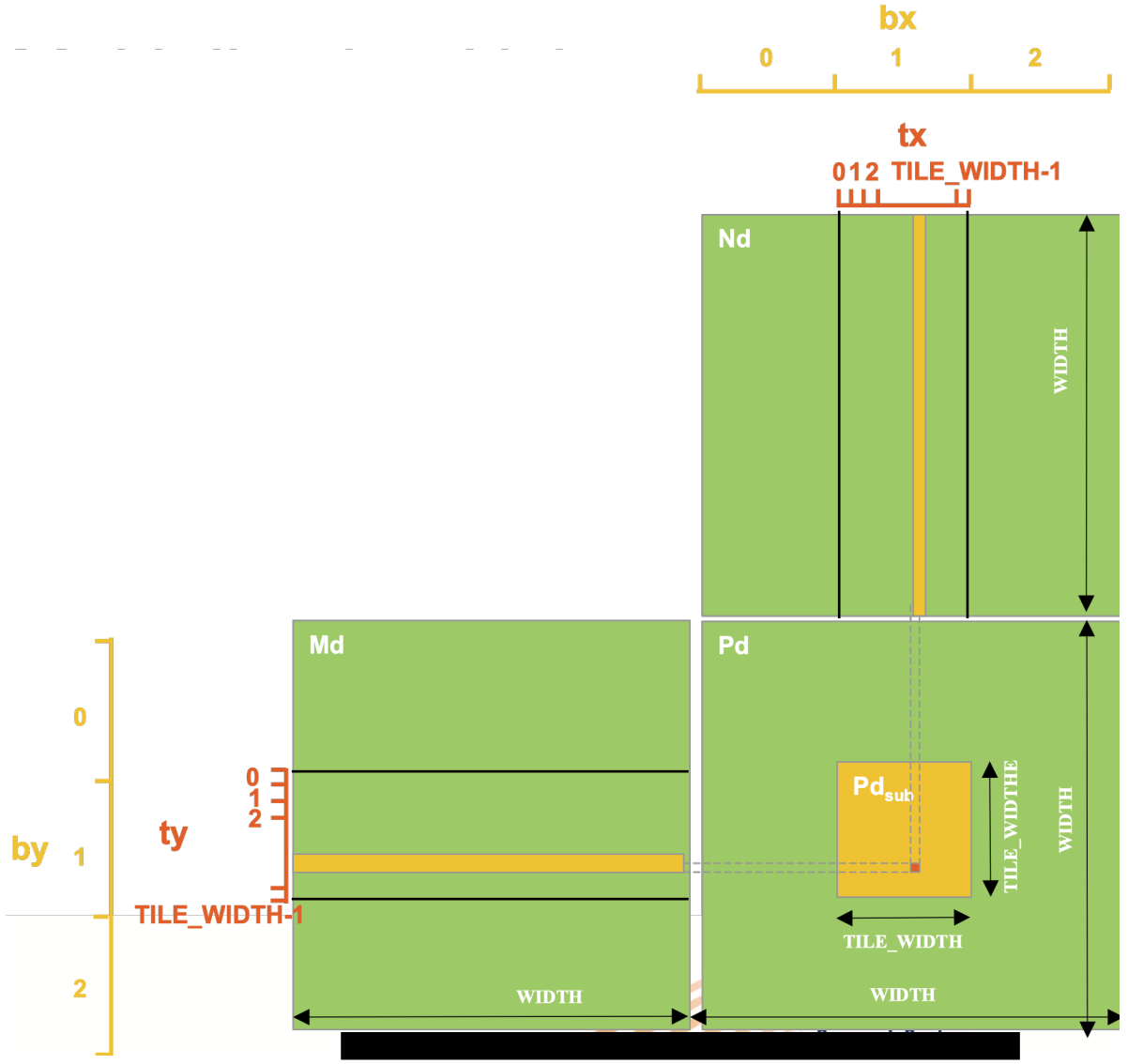
```
// Matrix multiplication on the (CPU) host in single
precision
void MatrixMulOnHost(float* M, float* N, float* P,
int Width)
{
    for (int i = 0; i < Width; ++i) {
        for (int j = 0; j < Width; ++j) {
            float sum = 0;
            for (int k = 0; k < Width; ++k) {
                float a = M[i * Width + k];
                float b = N[k * Width + j];
                sum += a * b;
            }
            P[i * Width + j] = sum;
        }
    }
}
```



Matrix Multiplication Using Multiple Thread Blocks

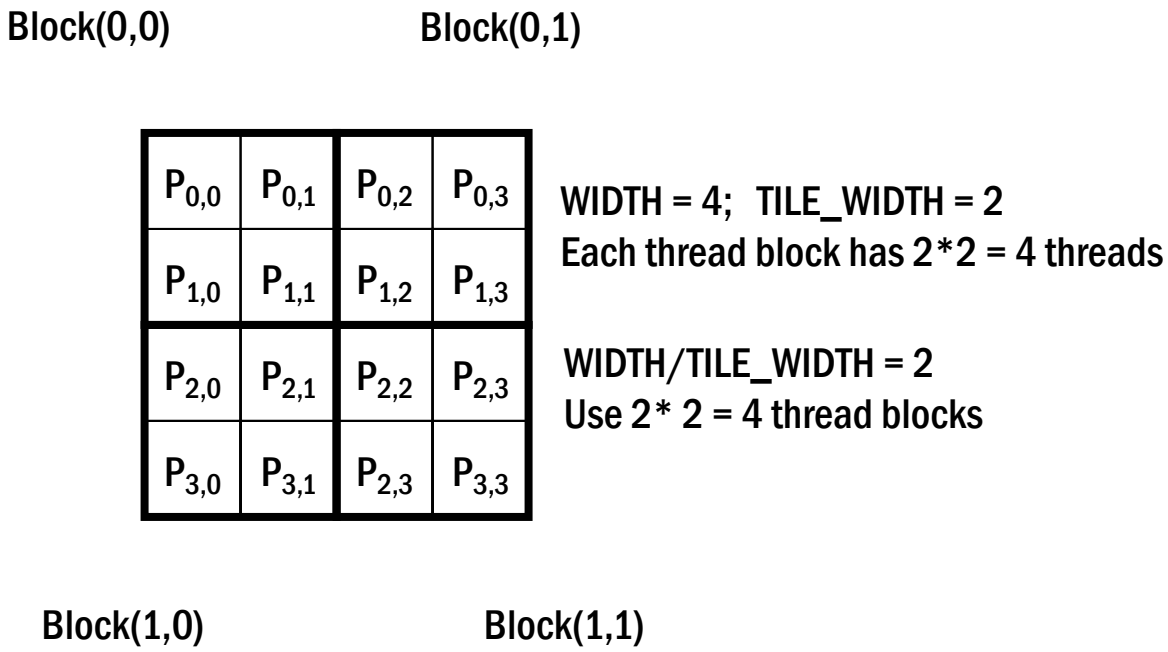
- **Partition Pd (output matrix) into tiles and map to thread blocks**
- Each thread block calculates one output tile
  - Each thread calculates one element
- Data access pattern
  - Each thread accesses a row of M and a column of N
  - Each thread block accesses a strip of M and a strip of N

  → Output tiling



CUDA Kernel: A Small Example

- Have each 2D thread block to compute a **(TILE_WIDTH)^2** tile of the result matrix **(P)**
  - Each has **(TILE_WIDTH)^2 threads**
- Generate a 2D index-space of **(WIDTH/TILE_WIDTH)^2** thread blocks



CUDA Kernel: A Bigger Example

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$	$P_{0,4}$	$P_{0,5}$	$P_{0,6}$	$P_{0,7}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$	$P_{1,4}$	$P_{1,5}$	$P_{1,6}$	$P_{1,7}$
$P_{2,0}$	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$	$P_{2,4}$	$P_{2,5}$	$P_{2,6}$	$P_{2,7}$
$P_{3,0}$	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$	$P_{3,4}$	$P_{3,5}$	$P_{3,6}$	$P_{3,7}$
$P_{4,0}$	$P_{4,1}$	$P_{4,2}$	$P_{4,3}$	$P_{4,4}$	$P_{4,5}$	$P_{4,6}$	$P_{4,7}$
$P_{5,0}$	$P_{5,1}$	$P_{5,2}$	$P_{5,3}$	$P_{5,4}$	$P_{5,5}$	$P_{5,6}$	$P_{5,7}$
$P_{6,0}$	$P_{6,1}$	$P_{6,2}$	$P_{6,3}$	$P_{6,4}$	$P_{6,5}$	$P_{6,6}$	$P_{6,7}$
$P_{7,0}$	$P_{7,1}$	$P_{7,2}$	$P_{7,3}$	$P_{7,4}$	$P_{7,5}$	$P_{7,6}$	$P_{7,7}$

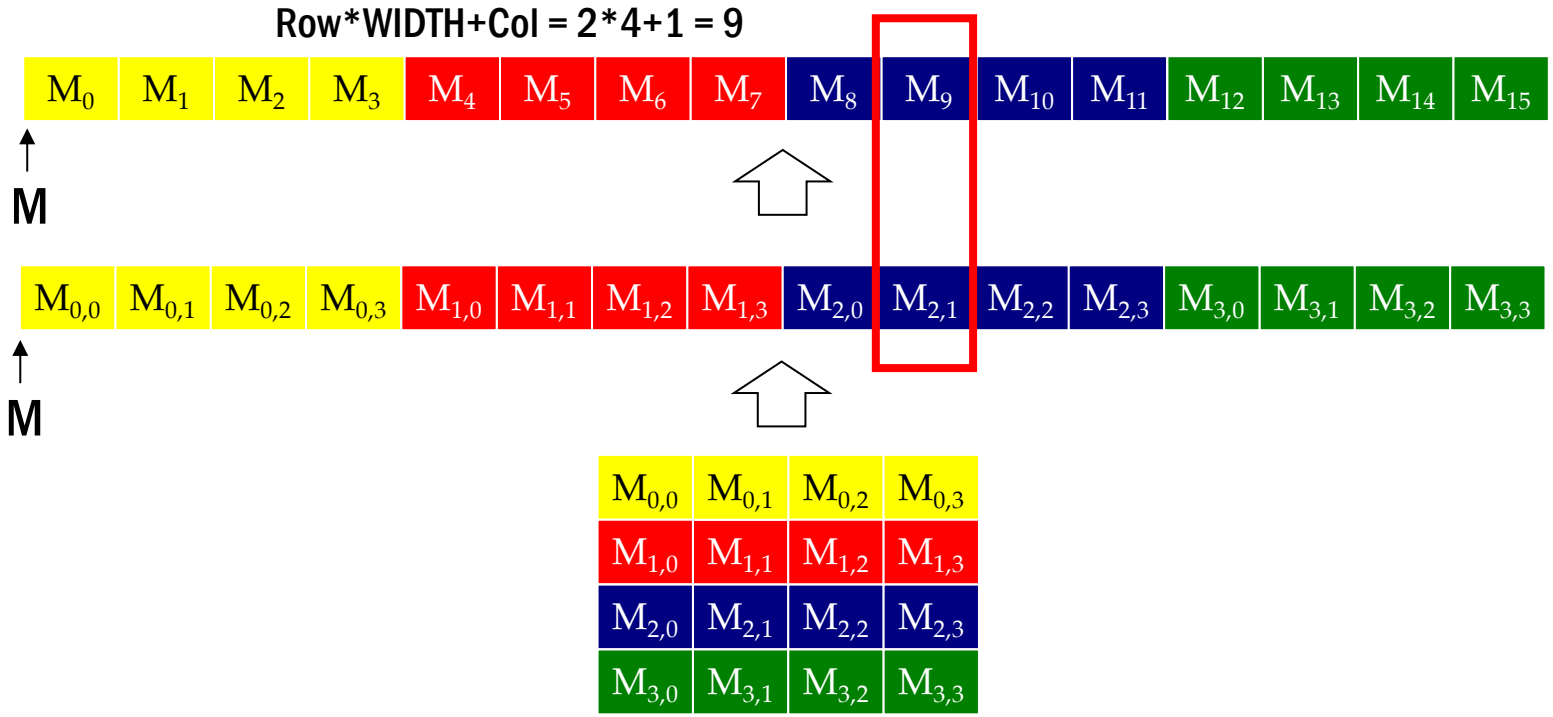
WIDTH = 8; TILE_WIDTH = 4
Each thread block has 4*4 =16 threads

WIDTH/TILE_WIDTH = 2
Use 2* 2 = 4 thread blocks

44

Memory Layout of Matrix in C/C++

- Local and shared arrays can be multi-dimensional in CUDA
- Global arrays in buffer objects need to be flattened into 1D arrays
  - Image objects can store 2 or 3-dimensional texture or image



Step 1: Input Matrix Data Transfer (Host)

```
void MatrixMulOnDevice(float* M, float* N, float* P, int Width) {
    int size = Width * Width * sizeof(float);
    cl_mem Md, Nd, Pd;
    cudaMalloc((void**) &Md, size);
    cudaMalloc((void**) &Nd, size);
    cudaMalloc((void**) &Pd, size);

    cudaMemCpy Md, M, size, cudaMemcpyHostToDevice);
    cudaMemCpy Nd, N, size, cudaMemcpyHostToDevice);
    cudaMemset Pd, 0, size);
```

Step 2: Output Matrix Data Transfer (Host)

```
// kernel invocation code
...

// Read P from the device
cudaMemCpy(P, Pd, size, cudaMemcpyDeviceToHost);

// Free device matrices
cudaFree(Md);
cudaFree(Nd);
cudaFree(Pd);
```

Step 3: CUDA Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {

  // Calculate the row index of the P element and M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

  // Calculate the column index of P and N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < Width; ++k) {
      Pvalue += M[Row*Width+k]*N[k*Width+Col];
    }
    P[Row*Width+Col] = Pvalue;
  }
}
```

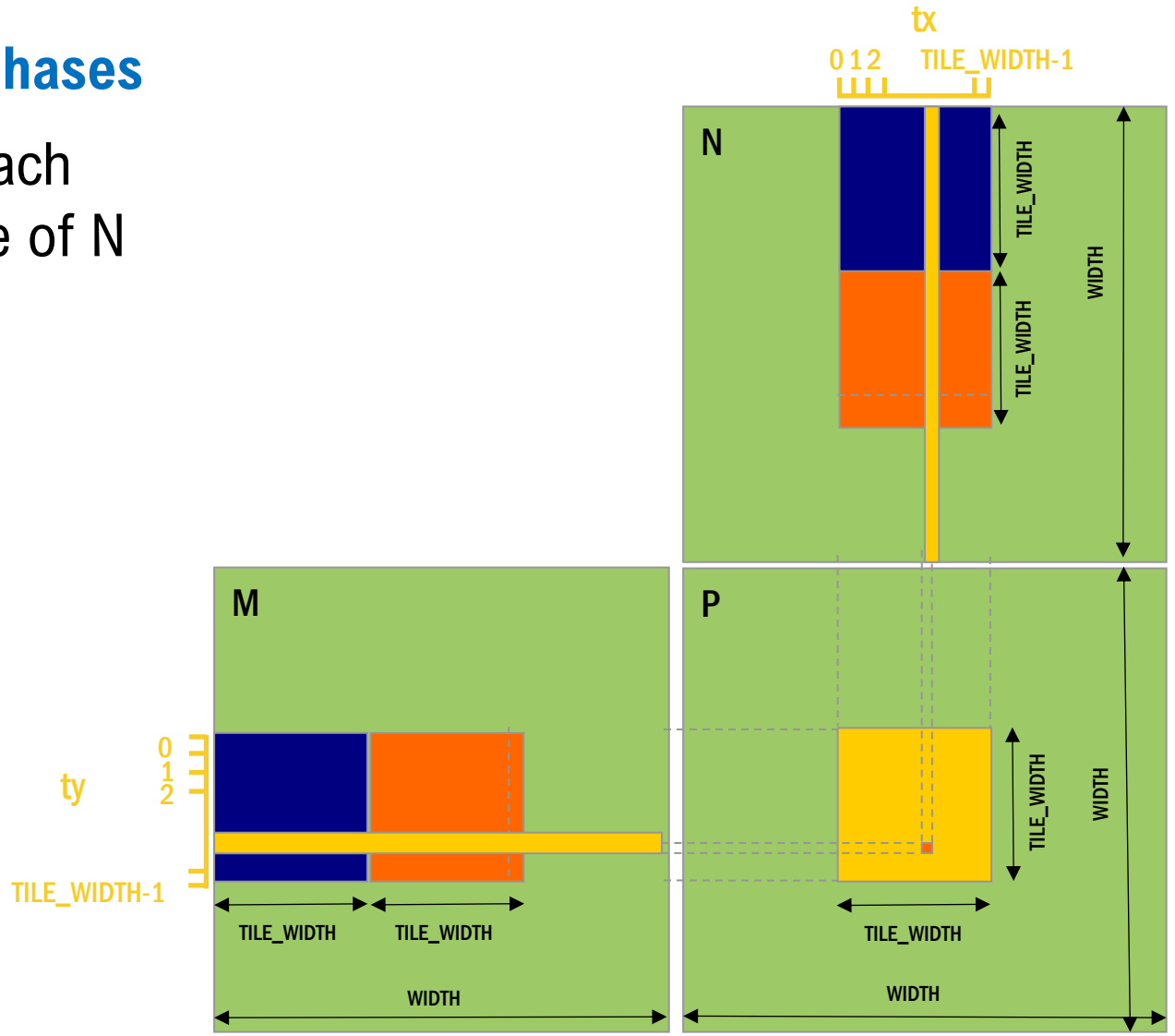
Tiled Matrix Multiplication

◎ Break up the execution of each thread into **phases**

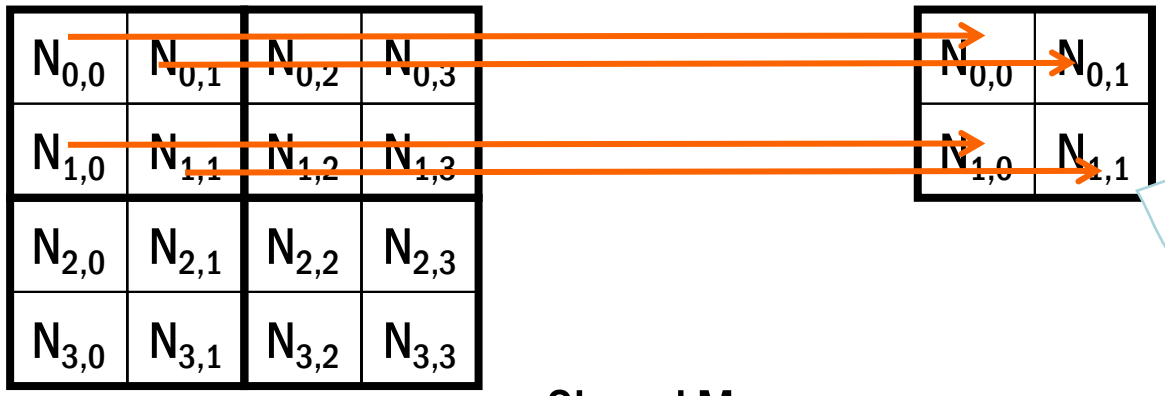
◎ So that the data accesses by the thread in each phase are focused on one tile of M and one tile of N

→ **Input tiling**

◎ The tile is of TILE_WIDTH elements in each dimension



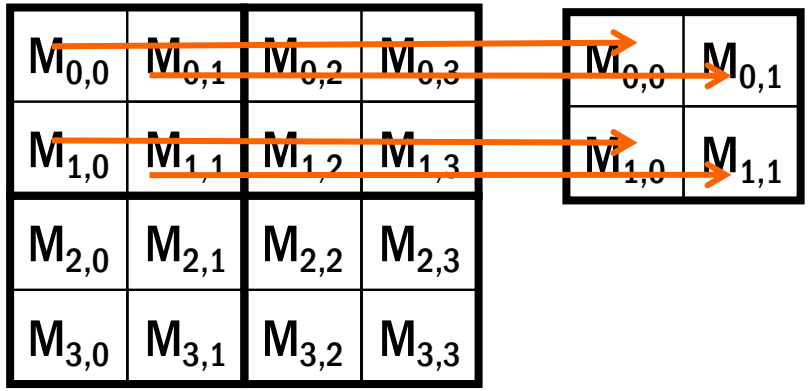
Phase 0: Load for P Block (0,0)

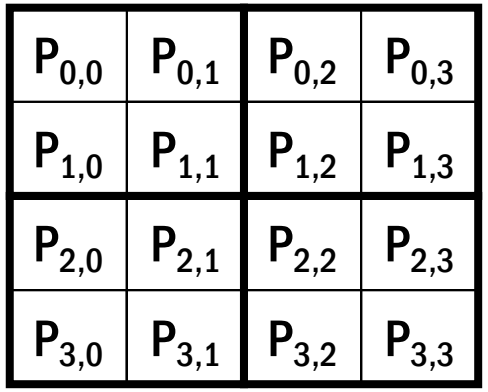


Shared Memory

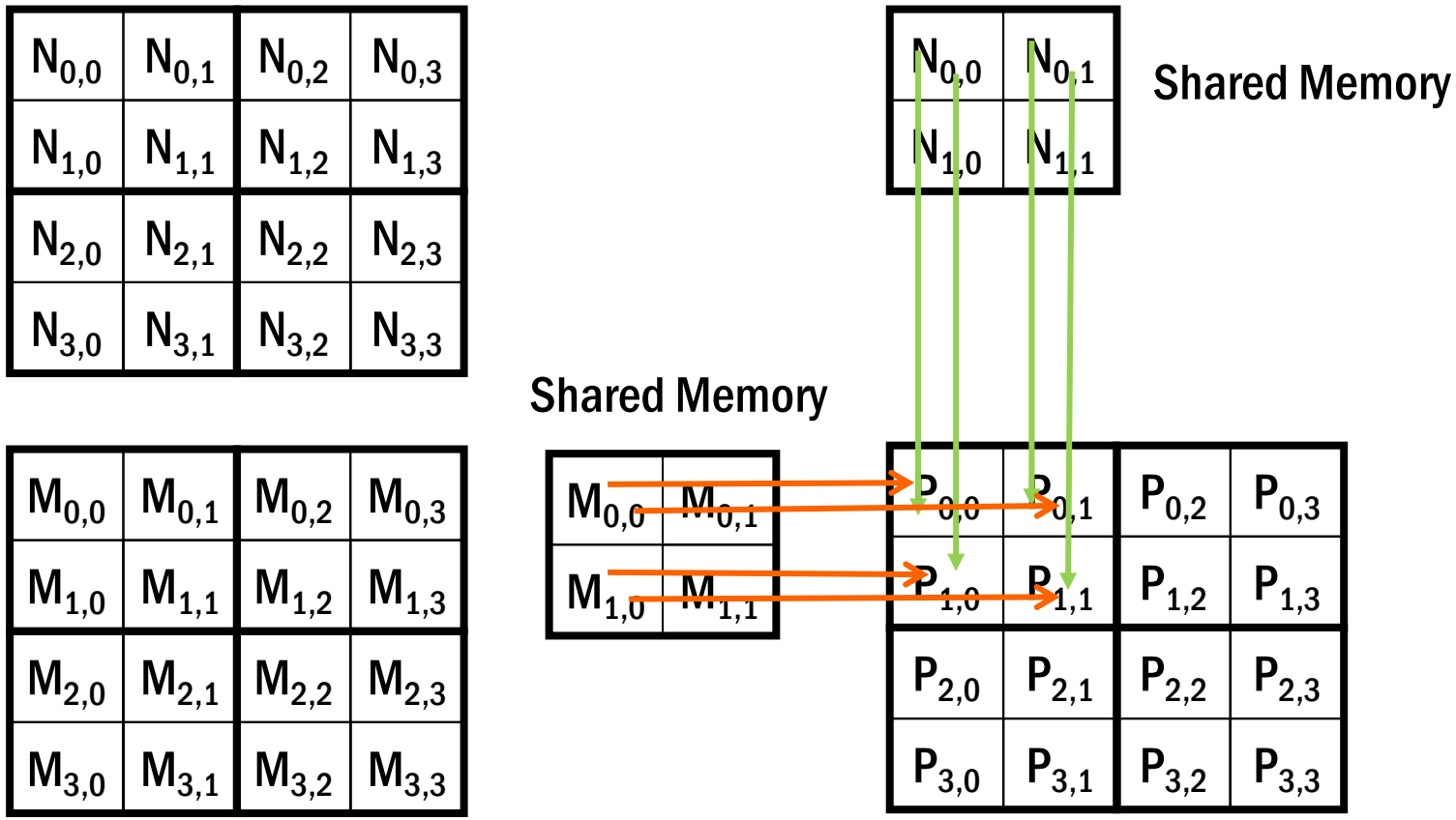
All threads in a thread block participate
Each thread loads one M element and one N element in tiled code

Shared Memory

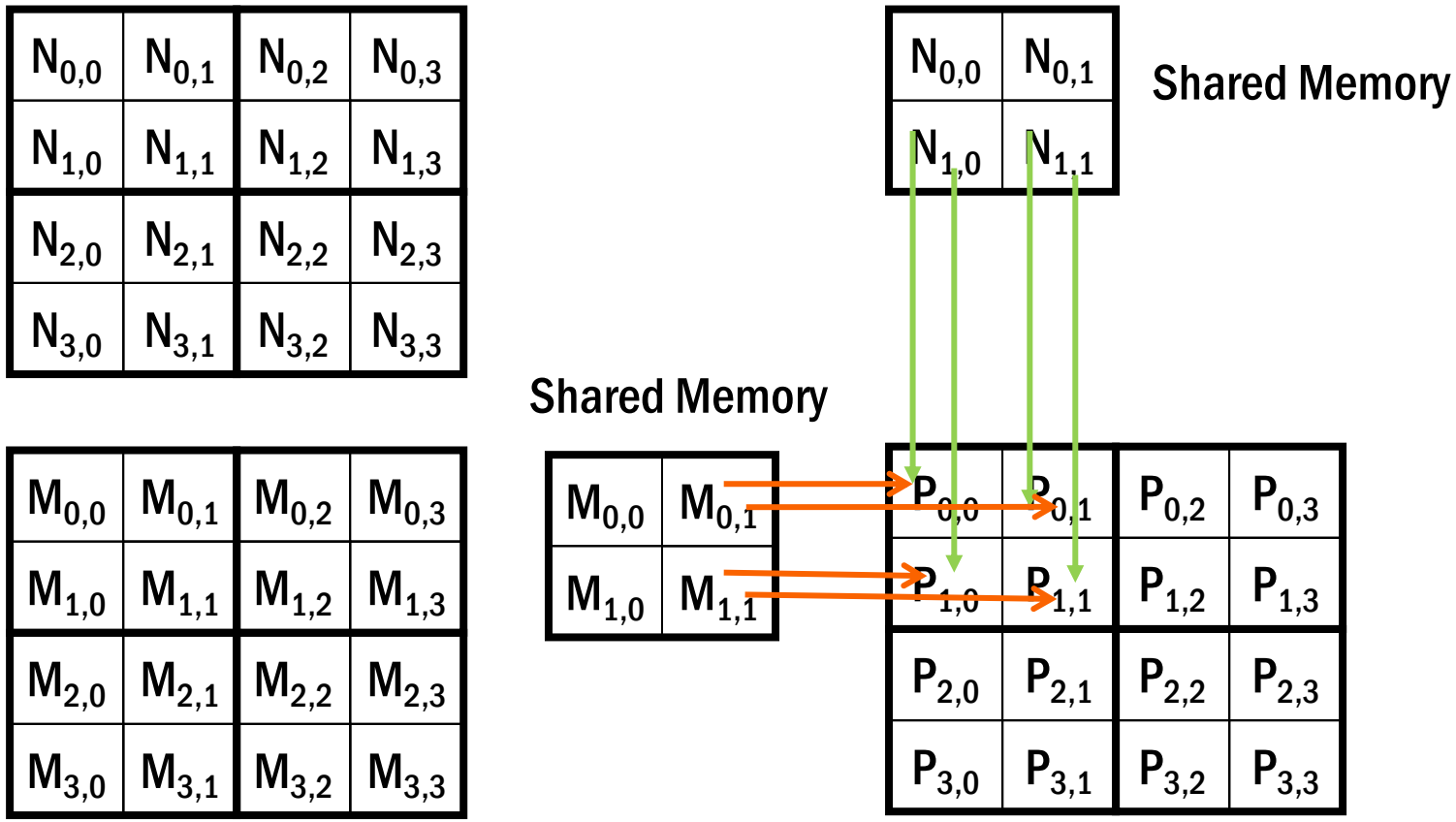




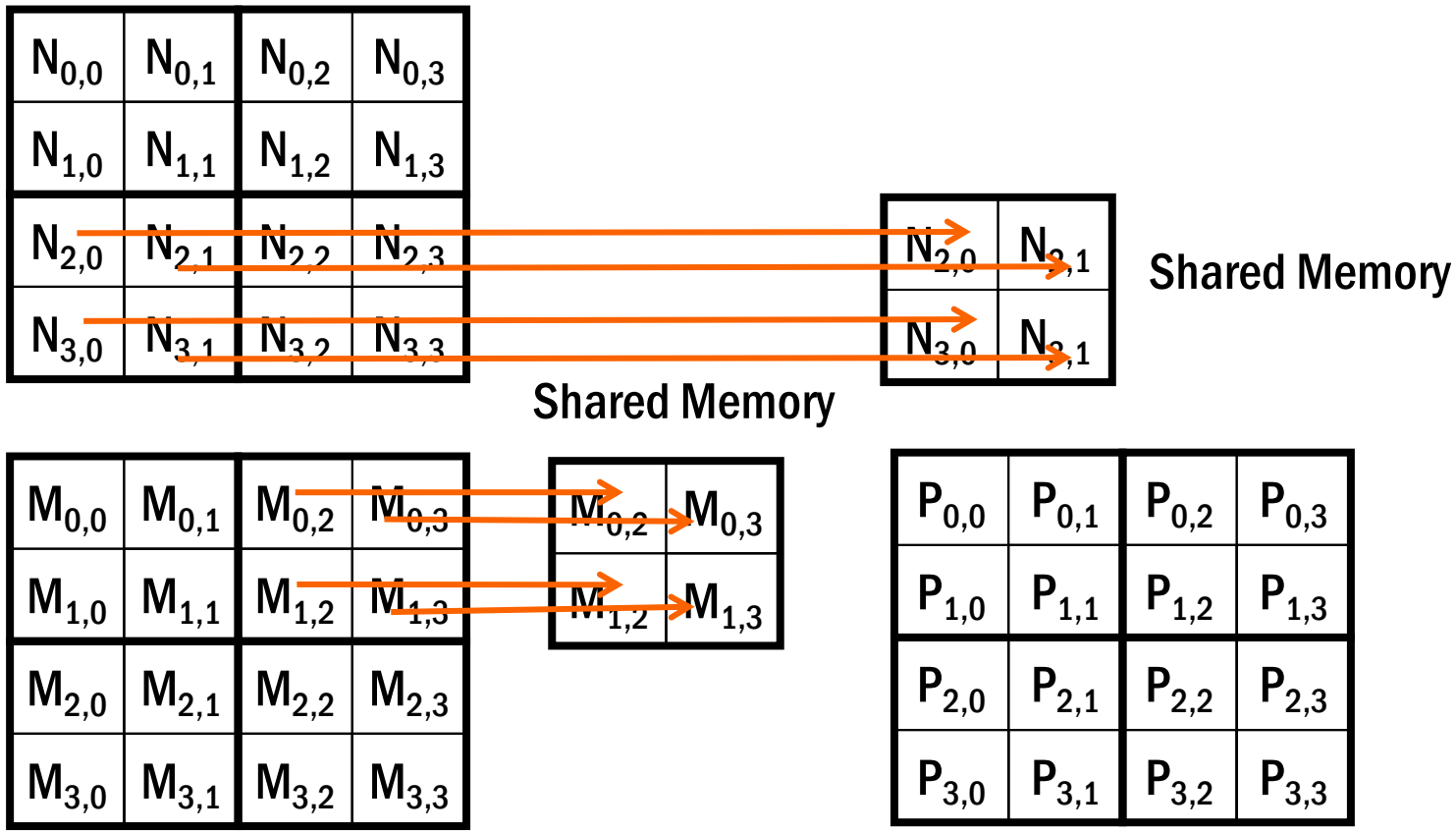
Phase 0: Use for P Block (0,0) (iteration 0)



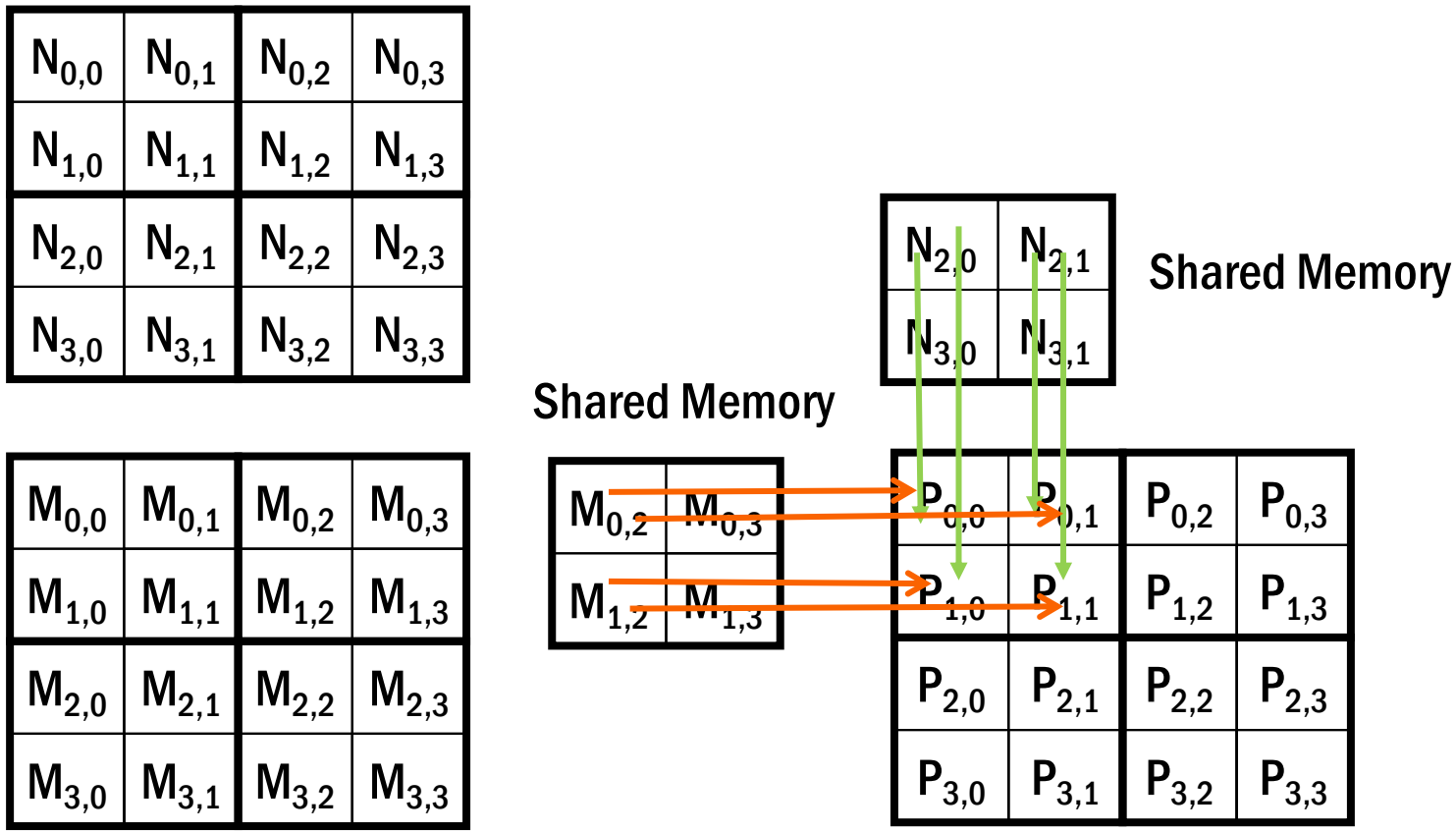
Phase 0: Use for P Block (0,0) (iteration 1)



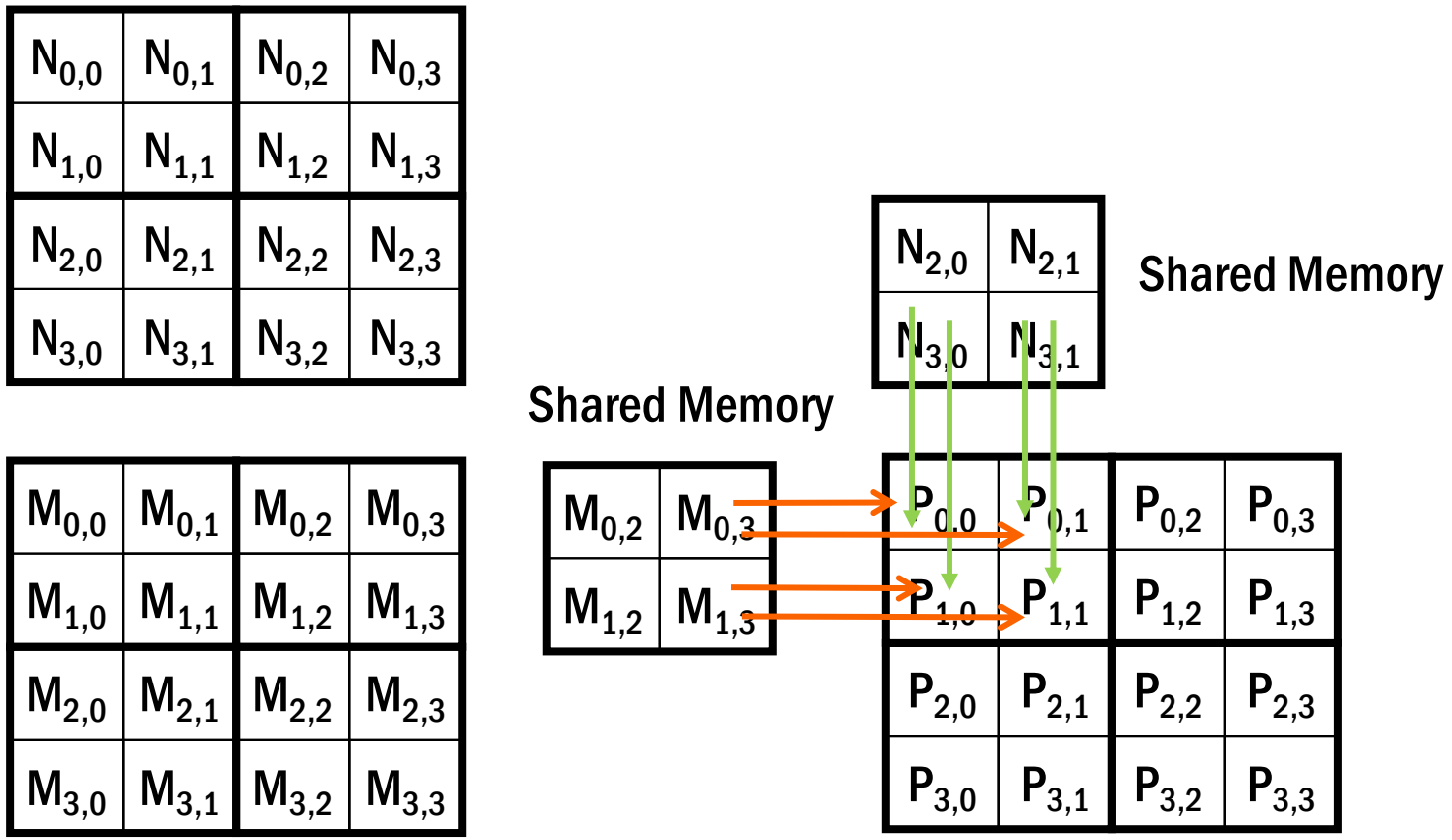
Phase 1: Load for P Block (0,0)



Phase 1: Use for P Block (0,0) (iteration 0)



Phase 1: Use for P Block (0,0) (iteration 1)



Loading Input Tiles

◎ Accessing tile m in 2D indexing:

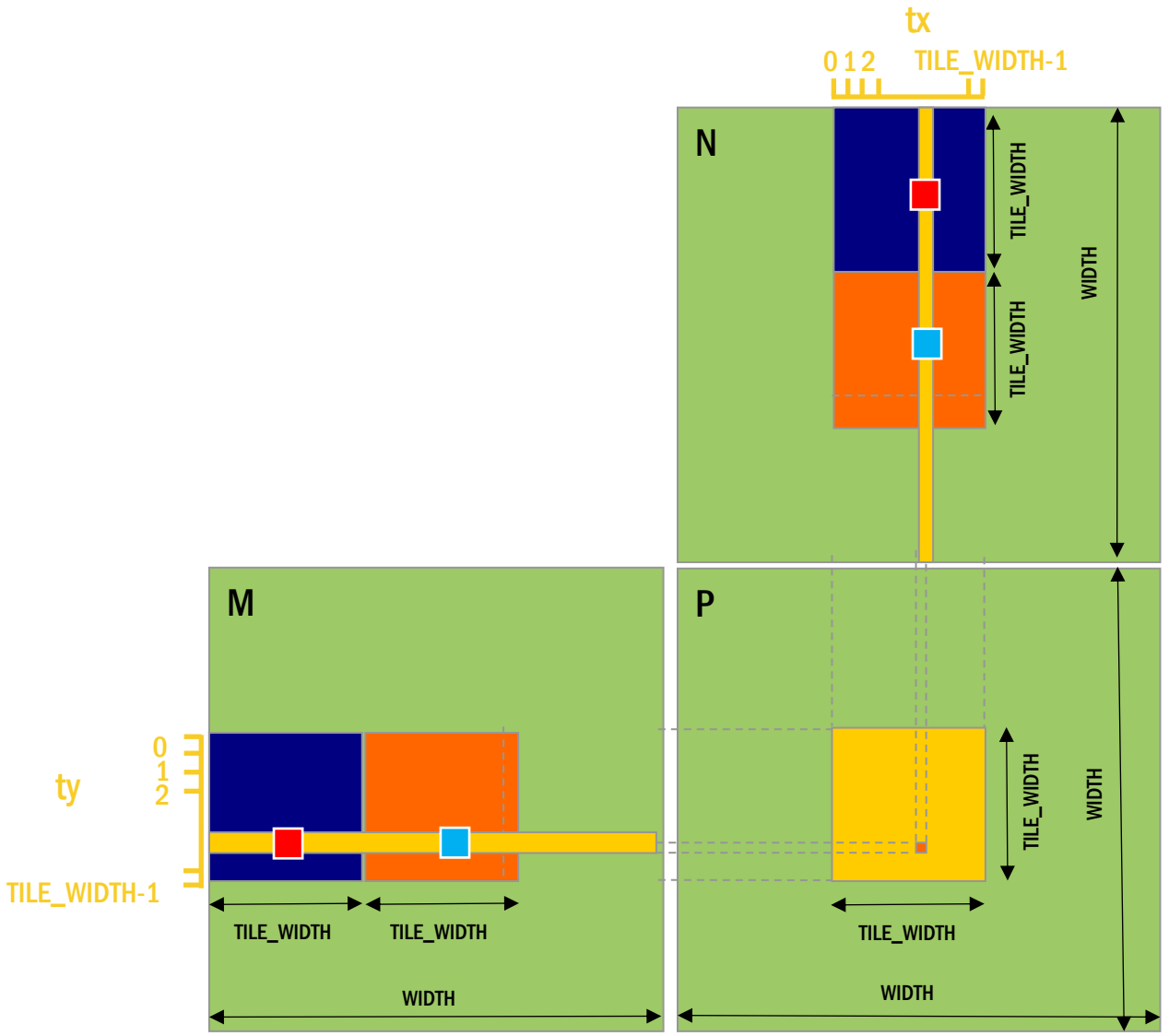
M[row][m*TILE_WIDTH+tx]

N[m*TILE_WIDTH+ty][col]

◎ Accessing tile m in 1D indexing:

M[row*width + m*TILE_WIDTH+tx]

N[(m*TILE_WIDTH+ty)*width + col]

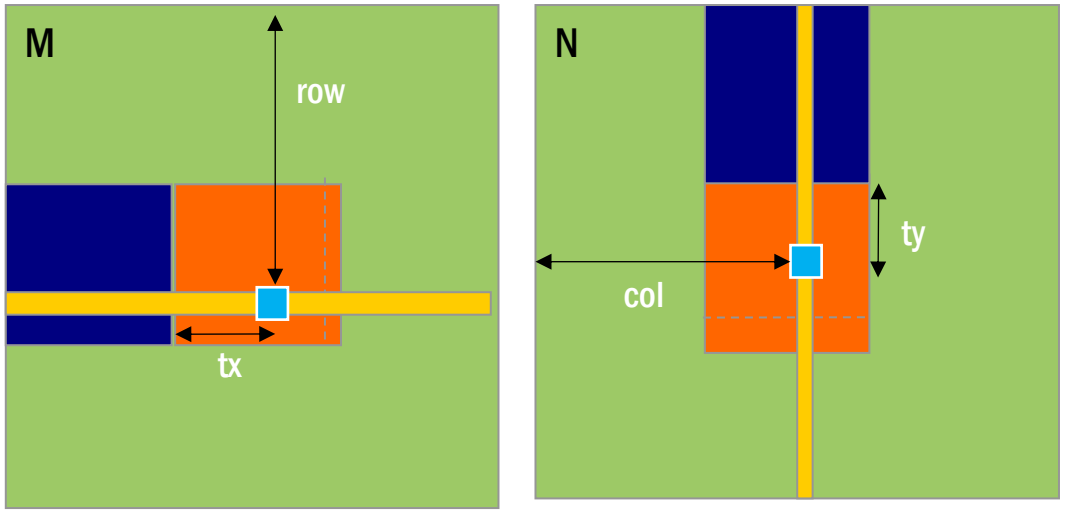


Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;
```



Tiled Matrix Multiplication Kernel

```
// Loop over the M and N tiles required to compute the P element
for (int p = 0; p < Width/TILE_WIDTH; ++p) {
   // Collaborative loading of M and N tiles into shared memory
   ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
   ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
   __syncthreads();

   for (int i = 0; i < TILE_WIDTH; ++i)
      Pvalue += ds_M[ty][i] * ds_N[i][tx];

   __synchthreads();
  }

  P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
// Loop over the M and N tiles required to compute the P element
for (int p = 0; p < Width/TILE_WIDTH; ++p) {
   // Collaborative loading of M and N tiles into shared memory
   ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
   ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
   __syncthreads();

   for (int i = 0; i < TILE_WIDTH; ++i)
      Pvalue += ds_M[ty][i] * ds_N[i][tx];

   __synchthreads();
  }

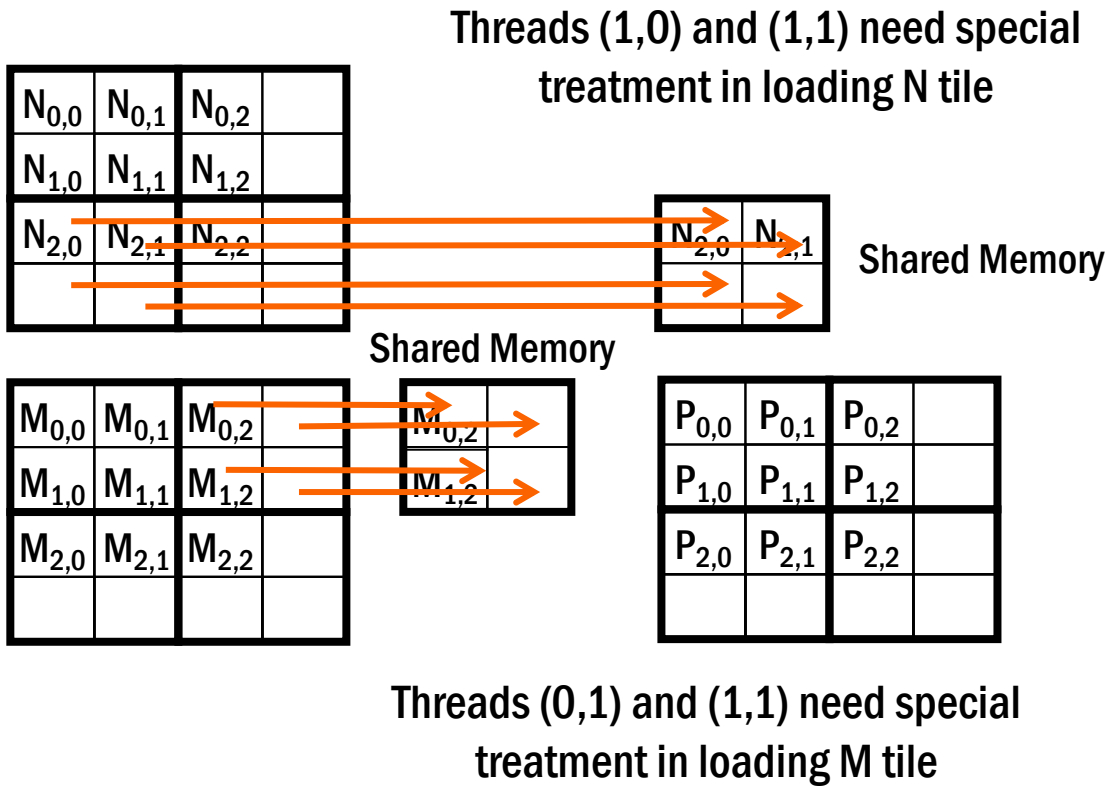
  P[Row*Width+Col] = Pvalue;
}
```

After all data is loaded into shared memory

After all data is read from shared memory

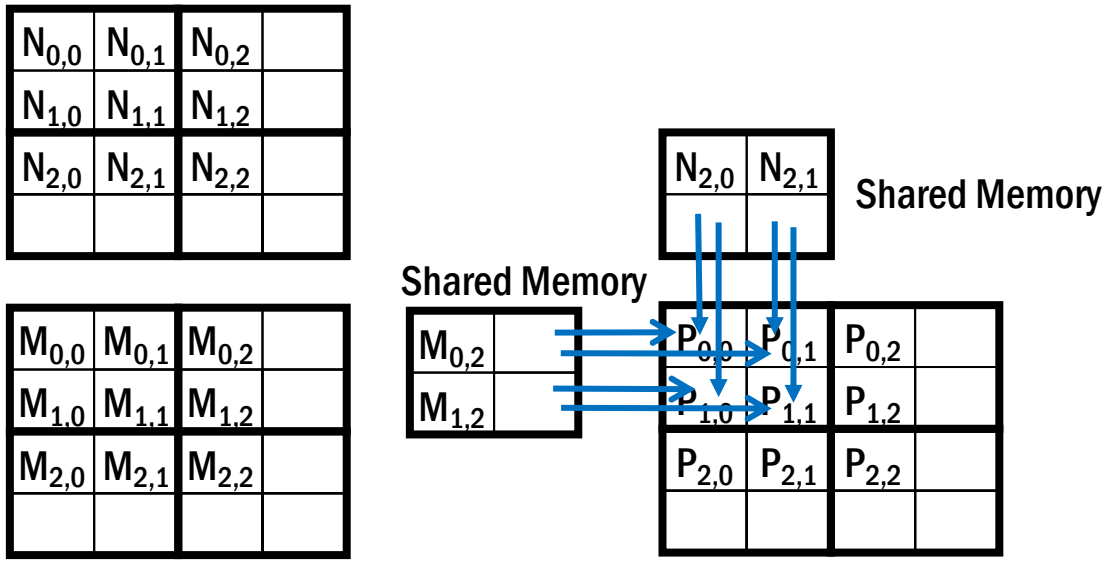
Handling Matrix of Arbitrary Size

◎ Matrices can have dimensions (width) that are not multiples of the tile width (TILE_WIDTH)



Handling Matrix of Arbitrary Size

◎ Matrices can have dimensions (width) that are not multiples of the tile width (TILE_WIDTH)



All Threads need special treatment. None of them should introduce invalidate contributions to their P elements.

Solution

◎ When a thread is to load any input element, test if it is in the valid index range

- If valid, proceed to load
- Otherwise, do not load and just write zero

◎ Rationale: a zero value will ensure that the multiply-add step does not affect the final value of the output element

◎ The condition tested for loading input elements is different from the test for calculating output P element

- A thread that does not calculate P element can still participate in loading input tile elements

Tile Size Consideration

◎ Each thread block should have many threads

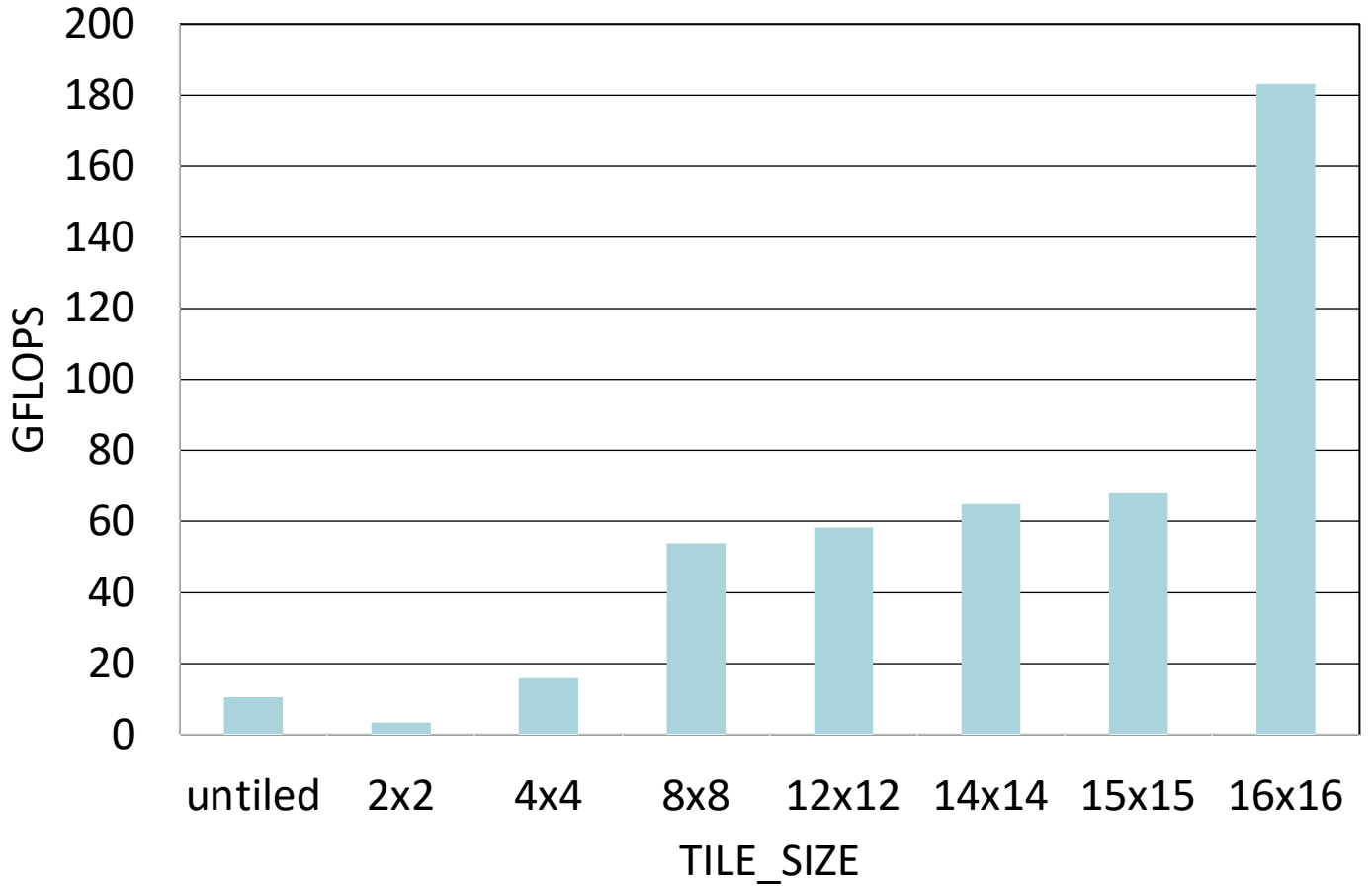
- TILE_WIDTH of 16 gives 16*16 = 256 threads
- TILE_WIDTH of 32 gives 32*32 = 1024 threads

◎ For 16, in each phase, each block performs 2*256 = 512 float loads from global memory for (2*16) * 256 = 8,192 mul/add operations (16 floating-point operations for each memory load or 0.25 B/op)

- C.f. 4 B/op from the naïve implementation

◎ For 32, in each phase, each block performs 2*1024 = 2048 float loads from global memory for (2*32) * 1024 = 65,536 mul/add operations (32 floating-point operation for each memory load or 0.125 B/op)

TILE_SIZE Effects



Will larger TILE_SIZE would result in even higher performance?

Shared Memory and Threading

- For an SM with 16KB shared memory
  - Shared memory size is implementation dependent!
  - For TILE_WIDTH = 16, each thread block uses 2*(16*16)*4B = 2KB of shared memory
    - Allowing 8 thread blocks active at the same time (256 threads per thread block)
    - This allows up to 8*2*256 = 4,096 pending loads (2 per thread, 256 threads per thread block)
  - For TILE_WIDTH = 32, each work group uses 2*(32*32)*4B = 8KB shared memory
    - Allowing 2 thread blocks active at the same time (1,024 threads per thread block)
    - However, in a GPU where the thread count is limited to 1536 threads per SM, the number of blocks per SM is reduced to one!
- Each barrier can reduce the number of active threads for a thread block
  - More thread blocks can be advantageous

Optimization Analysis

- Experiment performed on NVIDIA GT200 (2008)
- This optimization was clearly worth the effort

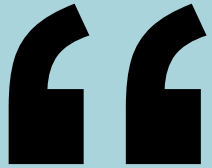
Implementation	Original	Improved
Global Loads	$2N^3$	$2N^2$ *(N/TILE_WIDTH)
Throughput	10.7 GFLOPS	183.9 GFLOPS
SLOCs (source lines of code)	20	44
Relative Improvement	1x	17.2x
Improvement/SLOC	1x	7.8x

More Recent Optimization Analysis

- Experiment on NVIDIA Tesla V100 (2017)
- Performance gain from tiling is less than 2x
  - L1 cache with larger capacity and higher bandwidth
    - 64KB L1+shared memory (GT200) vs 128KB L1+shared memory (V100)
  - Main memory with higher bandwidth
    - 100-150GB/s (GT200) vs. 900GB/s (v100)

Table 2. Performance Improvements Optimizing C = AB Matrix Multiply

Optimization	NVIDIA Tesla V100
No optimization	119.9 GB/s
Coalesced using shared memory to store a tile of A	144.4 GB/s
Using shared memory to eliminate redundant reads of a tile of B	195.5 GB/s



Next Class

Memory and Data Locality 1